CENTRO UNIVERSITÁRIO DO ESTADO DO PARÁ

ÁREA DE CIÊNCIAS EXATAS E TECNOLOGIA

BACHARELADO EM CIÊNCIA DA COMPUTAÇÃO

Cássio Roberto Saraiva de Oliveira

Felipe Fernandes da Silva

Renato Araújo Lima

VIRTUAL JUSTICE: JOGO EDUCACIONAL PARA ENSINO DE LINGUAGEM E TÉCNICAS DE PROGRAMAÇÃO ORIENTADA A OBJETOS

Belém

2013

CENTRO UNIVERSITÁRIO DO ESTADO DO PARÁ

ÁREA DE CIÊNCIAS EXATAS E TECNOLOGIA

BACHARELADO EM CIÊNCIA DA COMPUTAÇÃO

Cássio Roberto Saraiva de Oliveira

Felipe Fernandes da Silva

Renato Araújo Lima

VIRTUAL JUSTICE: JOGO EDUCACIONAL PARA ENSINO DE LINGUAGEM E TÉCNICAS DE PROGRAMAÇÃO ORIENTADA A OBJETOS

Trabalho de Curso na modalidade Produto, apresentado como requisito parcial para obtenção do grau em Bacharelado em Ciência da Computação do Centro Universitário do Estado do Pará – CESUPA, sob orientação do Professor Doutor Otávio Noura Teixeira e Co-orientação do M.Sc. Ricardo Melo Casseb do Carmo.

Belém

2013

Cássio Roberto Saraiva de Oliveira

Felipe Fernandes da Silva

Renato Araújo Lima

VIRTUAL JUSTICE: JOGO EDUCACIONAL PARA ENSINO DE LINGUAGEM E TÉCNICAS DE PROGRAMAÇÃO ORIENTADA A OBJETOS

Trabalho de Curso na modalidade Produto apresentado como requisito parcial para obtenção do grau em Bacharelado em Ciência da Computação do Centro Universitário do Estado do Pará – CESUPA

Data da Defesa: 09/12

Prof. Orientador Dr. Otávio Noura Teixeira - CESUPA

Prof. Co-orientador M.Sc. Ricardo Melo Casseb do Carmo - CESUPA

Prof. M.Sc. Igor Ruiz Gomes - CESUPA

Belém

2013

AGRADECIMENTOS

Agradeço primeiramente a minha mãe por ter dado para mim essa oportunidade de fazer um curso de nível superior, por ter passado a sua confiança em momentos difíceis tanto da minha vida pessoal quanto acadêmica, por todas as chamadas de atenções ao longo da minha vida que me colocaram de volta no trilho e por tudo que ela fez de bom ou de ruim, porque caso contrário eu não estaria escrevendo esse agradecimento.

Agradeço também a todas as minhas tias e minha finada avó por parte de pai que me ajudaram a trilhar esse caminho sempre me passando força junto com a minha mãe. Agradeço a meus parceiros e amigos Renato Lima e Felipe Fernandes, que juntos podemos viver essa experiência incrível que eu espero continuar vivendo por um longo tempo.

Agradeço ao corpo docente do meu curso e meus orientadores, Otávio Noura Teixeira e Ricardo Melo Casseb que aumentaram minha autoestima com conversas e incentivos positivos. Agradeço também a Instituição (CESUPA), por fornecer o equipamento necessário para a minha progressão nesse curso. Muito obrigado a todos que me ajudaram.

Cássio Roberto Saraiva de Oliveira

AGRADECIMENTOS

Agradeço primeiramente a DEUS pela vida e minha trajetória. A minha família de modo geral que sempre esteve comigo me apoiando, em especial aos meus pais Rose Mary de Lourdes Ferreira da Silva e Antônio Marcelino Fernandes da Silva.

A instituição e todos os seus profissionais, agradeço aos nossos professores amigos e orientadores, Ricardo Melo Casseb e Otávio Noura Teixeira além de nossa professora Andrea Cristina Araújo que juntos sempre se demonstraram presentes para resolver nossas duvida e nos auxiliar em nosso trabalho, obrigado Professores.

Agradeço a todos os meus amigos, e em especial meus companheiros nesse trabalho Cássio Roberto Saraiva e Renato Araújo Lima, sempre apoiando e tentando me tranquilizar.

Felipe Fernandes da Silva

AGRADECIMENTOS

Gostaria de agradecer a DEUS por ter me dado a oportunidade de poder fazer aquilo que gosto e fazê-lo bem feito, além de me guiar e proteger. Dedico meus agradecimentos a minha família, principalmente a meus pais Jorge Antônio Carneiro Lima e Maria de Nazaré Araújo Lima, minha irmã, minhas avós e meus avôs.

Agradeço a todo o corpo docente do CESUPA por ter me acompanhado durante os 4 anos que estive na instituição, me ensinando e orientando, em especial aos professores Otávio Noura Teixeira, Ricardo Melo Casseb e Igor Ruiz Gomes que foram não só professores, mas grandes amizades que formei na instituição. Agradecimento único a professora Alessandra Natasha Barreiros por ter me arrastado para várias atividades que me deixavam exausto, mas que no fim eram recompensadores.

Para finalizar, agradeço a todos que contribuíram para o desenvolvimento do projeto trabalhando em atividades que exigiam uma habilidade que não competia com a nossa. Thatiana Ferreira de Barros e Regina Mares de Souza Oliveira, que além de amigas possuem belas vozes para as dublagens.

Hélio Souza que trabalhou na gravação e produção dos áudios utilizados no jogo e Neil Turnbull – *thanks for the help with the music and hope we could work again another time*. É claro que não posso deixar de agradecer a meus colegas de trabalho Cássio Roberto e Felipe Fernandes. Acredito no sucesso de cada um e também percebemos que temos um bom trabalho em equipe. Espero muito que sigamos nossos sonhos e alcancemos o sucesso.

Renato Araújo Lima

“I think everyone in this country should learn to program a computer. Everyone should learn a computer language because it teaches you how to think. I think of computer science as a liberal art”.

Steve Jobs - Lost Interview, 1996.

RESUMO

O ensino de linguagens e técnicas de programação é muito útil para o desenvolvimento do pensamento lógico e abstração de qualquer tipo de problema, para se passar tal conhecimento, uma metodologia de ensino alternativa foi utilizada, um jogo educacional. Baseado em teorias construtivistas e conceitos de game design, foi criada a ideia de um jogo que busque o interesse do jogador pelo conhecimento e o prazer em jogar. Para desenvolver o produto foi feita a modelagem, animação e texturização 3D dos objetos, composição de recursos sonoros, desenvolvimento de uma linguagem de programação única e a integração de todos com o jogo objetivando passar a compreensão da programação de maneira dinâmica e agradável.

Palavras-chave: Virtual Justice. Jogo Educacional. Construtivismo. Linguagem e Técnicas de Programação. Orientação a Objetos. Game Design. Interpretador. Unity. Blender.

ABSTRACT

The programming languages and techniques teaching is very useful for the development of the logical thoughts and abstraction of any kind of problem, to pass such knowledge, an alternative teaching methodology was used, an educational game. Based on constructivist theories and game design concepts, it was created the idea of a game that looks for the player's interest by the knowledge and joy for playing. To develop the product, was made the modeling, animation and texturing of 3D objects, sound resources composing, development of a unique programming language and the interaction of all with the game aiming to pass the comprehension of programming in a dynamic and enjoyable way.

Keywords: Virtual Justice. Educational game. Constructivism. Programming Language and Techniques. Object Orientation. Game Design. Interpreter. Unity. Blender.

LISTA DE FIGURAS

Figura 1: Zona de Desenvolvimento Proximal 9
Figura 2: Faixa Etária de Jogadores de *Videogame* de 2013 nos Estados Unidos 13
Figura 3: Pirâmide Etária Mundial de 2010 14
Figura 4: Exemplificação da motivação de *Assassin's Creed 2* 15
Figura 5: 5 Grandes Fatores aplicados ao Virtual Justice 16
Figura 6: Astra.IA 18
Figura 7: Representação dos elementos presentes no jogo 19
Figura 8: Exemplo da primeira versão do código executado pela VJL 26
Figura 9: Representação da antiga locação de memoria para a VJL 27
Figura 10: Representação da atual locação de memória para a VJL 28
Figura 11: Atual sintaxe da linguagem VJL 29
Figura 12: Diagrama de Caso de Uso para o interpretador VJL 31
Figura 13: Classes para gerenciamento de recursos 32
Figura 14: Classes para análise de códigos VJL 34
Figura 15: Classes responsáveis pela interpretação dos comandos VJL 36
Figura 16: Diagrama de Atividade do interpretador VJL 38
Figura 17: Inimigo ‘Tank’, arte conceito e modelo respectivamente 40
Figura 18: Aplicação de *Mark Seam* no modelo 40
Figura 19: UV pronto para a aplicação da textura 41
Figura 20: *Ambient Occlusion* e *Point Light Map* respectivamente 41
Figura 21: Textura e modelo final 42
Figura 22: Esqueleto do modelo 42
Figura 23: Ambiente de animação em *Blender* 43
Figura 24: Barra de vida 44
Figura 25: IDE 44
Figura 26: Janela de tutoriais 45

Figura 27: Espada do estoque renderizada por Cycles 46
Figura 28: Pistola do estoque renderizada por Cycles 46
Figura 29: Objeto explicativo para janela de tutorial – Memória RAM 47
Figura 30: Combate com espada 48
Figura 31: Modo FPS 49

LISTA DE SIGLAS

FPS	First Person Shooter
IDE	Integrated Development Environment.
RPG	Role Playing Game.
TIC	Tecnologia da Informação e Computação.
UML	Unified Modeling Language.
VJL	Virtual Justice Language.

SUMÁRIO

INTRODUÇÃO **1**

PROBLEMA 2

OBJETIVO GERAL 2

OBJETIVOS ESPECÍFICOS 2

JUSTIFICATIVA 3

METODOLOGIA DE DESENVOLVIMENTO 3

1 BASE TEÓRICA **5**

1.1 CONSTRUTIVISMO DE JEAN PIAGET 5

1.2 CORRENTE SOCIOCONSTRUTIVISTAS DE VYGOTSKY 7

1.3 JOGOS EDUCACIONAIS 9

2 CONCEPÇÃO DA IDEIA **12**

2.1 PÚBLICO-ALVO 12

2.2 OS 5 GRANDES FATORES 14

2.3 EXPERIÊNCIA DE JOGO 17

3 TECNOLOGIAS UTILIZADAS **20**

3.1 ASTAH COMMUNITY 20

3.2 UNITY 3D 20

3.3 DETONATOR 21

3.4 BLENDER 21

3.5 IDE'S 21

3.5.1 Microsoft Visual Studio 2010 Express **22**

3.5.2 MonoDevelop **22**

3.6 SOUND FORGE PRO 10 22

3.7 EDITORES GRÁFICOS 22

3.7.1 CorelDRAW X6 ..... **23**

3.7.2 Adobe Photoshop CS5 ..... **23**

3.7.3 Paint .NET **23**

4 DESENVOLVIMENTO DO VIRTUAL JUSTICE **24**

4.1 VISÃO 24

4.2 VIRTUAL JUSTICE LANGUAGE 24
4.2.1 Interpretador **24**
4.2.2 Escopo 25
4.2.3 Desenvolvimento do Interpretador VJL 25
4.2.4 Diagramas 30
4.2.4.1 Diagramas de Caso de Uso 30
4.2.4.2 Diagramas de Classes 31
4.2.4.3 Diagrama de Atividade 37
4.3 O JOGO 39
4.3.1 Elementos Visuais 39
4.3.1.1 Modelos 3D 39
4.3.1.2 Texturização 40
4.3.1.3 Animação 42
4.3.1.4 Interface Gráfica. 43
4.3.2 Sistema de Atributos e Itens 47
4.3.3 Lógica de Combate 48
4.3.4 Cenas Especiais, Legenda e Dublagem 49
4.3.5 Musicalidade 49
5 CONCLUSÃO 50
5.1 DIFICULDADES ENCONTRADAS 50
5.2 PROPOSTAS FUTURAS 51
REFERÊNCIAS 53
APÊNDICE A – MANUAL DE DESENVOLVIMENTO VJL 58
APÊNDICE B – Detalhamento de Caso de Uso para Análise Léxica. 70
APÊNDICE C – Detalhamento de Caso de Uso para Análise Semântica. 71
APÊNDICE D – Detalhamento de Caso de Uso para Análise Sintática. 72
APÊNDICE E – Detalhamento de Caso de Uso para Executar O Programa. 73
APÊNDICE F – Detalhamento de Caso de Uso para Validar o Código. 74
APÊNDICE G – Detalhamento de Caso de Uso para Visualizar os Erros. 75

INTRODUÇÃO

Tudo que o homem possui ou já possuiu para aperfeiçoar seu modo de viver se chama tecnologia. Como exemplo na educação, a tecnologia sempre esteve sendo utilizada para conseguir passar com mais clareza e eficiência informações importantes para o conhecimento do aluno.

Por mais que pareça ser a solução para todos os problemas, na realidade, observa-se que a tecnologia ainda é vista como um recurso dispensável para a maioria dos professores e instituições de ensino no Brasil. Por certas vezes, a tecnologia também é atacada como a principal causa do baixo rendimento de alunos e professores. Essas afirmações são apresentadas de maneira equivocada já que o processo de aprendizado se manifesta através de uma metodologia de ensino.

O processo de aprendizagem é uma tarefa cognitiva complexa e isso exige um esforço do aluno que está tentando realizá-la, logo o estudante precisa ter motivação para conseguir aprender. Por esse motivo a criação de um software educacional é de grande valia, pois em muitos casos é a motivação necessária que faltava ao estudante (VIRVOU; KATSIONES; MANOS, 2005).

O projeto começou com uma pequena versão que permite idealizar o produto como será apresentado neste trabalho. Um jogo educacional diferente, que ensina de maneira agradável um dos campos do conhecimento que recentemente vem sendo integrado aos planos de ensino nas escolas de outros países.

Segundo Gros (2007, p. 6, tradução nossa) "*videogames* são muito úteis para a aprendizagem de estratégias específicas e também para adquirir conhecimento de forma geral". Procurando-se entender melhor como jogos tem eficiência no ensino, o jogo foi projetado para que alcançasse a experiência mais estimulante possível visando o interesse do jogador em aprender a programar. No final, seria possível observar que a metodologia utilizada no trabalho desperta um interesse maior dos jogadores do que modelos anteriores a ele e avaliar se o conhecimento passado foi inteiramente absorvido.

PROBLEMA

A falta de metodologias alternativas de ensino para a educação dos brasileiros, em especial jogos educacionais, expõe o comportamento invariável da educação em relação a novas propostas e pesquisas na área de metodologias educativas.

Outra problemática é a ausência de campanhas e atividades que incentivem um estudo mais atencioso de conhecimentos da área de Tecnologia da Informação e Computação (TIC). Essa área do conhecimento já começa a ser explorada em vários países como um conhecimento fundamental para seus futuros cidadãos cabendo a eles decidirem que profissão escolher, mas independente disso terem conhecimentos ligados à informática.

Pelo fato de não serem ensinadas ou apresentadas de maneira eficiente a crianças e jovens, acaba não sendo criada uma comunidade de profissionais suficiente para a demanda de mercado. Esse fator acaba desacelerando o avanço tecnológico brasileiro e até mesmo o econômico que exige agora profissionais cada vez mais qualificados.

OBJETIVO GERAL

O objetivo principal é apresentar um jogo educacional que ensine programação básica e orientada a objetos de uma maneira diferente da qual outros jogos educacionais procuram passar o conhecimento.

OBJETIVOS ESPECÍFICOS

- Buscar a base teórica para a fundamentação pedagógica que o projeto deve ter para o desenvolvimento do jogo.
- Definir os recursos e arquitetura que serão utilizadas para o desenvolvimento do software.
- Desenvolver a linguagem de programação que estará contida dentro do jogo.
- Construir uma fase completa para a apresentação do produto com um início, meio e fim.

JUSTIFICATIVA

Conforme a tecnologia se torna mais presente na vida das pessoas, mais visível é a necessidade de profissionais que entendam os fundamentos dessa ferramenta. Aprender programação de software tornou-se há poucos anos um possível tópico a ser ensinado em escolas e instituições de ensino de fora do Brasil. Na Estônia, 100% dos alunos de 7 a 8 anos de idade de escola pública tem conhecimento de programação básica (DELL, 2012) não sendo o único país a participar de uma mobilização para a inserção da matéria de programação na grade escolar infanto-juvenil.

'Todos deveriam aprender a programar um computador, porque ele ensina como pensar'. Esta é uma das 50 frases de personalidades reconhecidas no mundo em um dos mais acessados sites de ensino de programação online, o Code.org[1]. Organizações como estas, apoiadas por governos e organizações preocupadas com o mercado de Tecnologia da Informação e Computação (TIC), procuram incentivar os jovens a aprenderem voluntariamente essa área do conhecimento.

Aproveitando o grande público que está crescendo interessado pela programação e a carência de metodologias similares no sistema público de ensino brasileiro, o jogo tem um diferencial que comprovadamente atrai milhares de jovens brasileiros. Segundo pesquisa divulgada no site do SEBRAE (2013) o mercado de jogos digitais movimentou R$ 5,3 bilhões em 2012 e teve um crescimento de 32% em relação ao ano anterior, e até 2017, jogos serão uma das principais tecnologias utilizadas na educação dos jovens brasileiros.

A metodologia de ensino ainda é bastante tradicional nas escolas brasileiras, entretanto o projeto entende essa situação e procura alcançar apenas o jovem como uma fonte de diversão e metodologia de ensino alternativa sendo até possível que o jogador procure o produto mais como uma forma de prazer do que de educação.

METODOLOGIA DE DESENVOLVIMENTO

O trabalho se organiza de maneira similar ao avanço nas etapas da metodologia que o projeto utilizou para seu desenvolvimento do produto. No capítulo 1, toda a base teórica buscada no campo da pedagogia foi definida para o apoio nas decisões que serão tomadas no capítulo 2, a concepção da ideia, este apresentará o direcionamento que será utilizado para o

[1] Site americano não governamental que promove e disponibiliza o ensino de programação gratuitamente (CODE, 2013, online).

desenvolvimento e balanceamento dos elementos para que o jogo em geral produza a experiência desejada nos jogadores. Depois de apresentadas no capítulo 3 as tecnologias que serão utilizadas no desenvolvimento do produto, que será em si descrito no capítulo 4. No quinto capítulo é descrito as considerações finais a respeito do trabalho, apresentando as conclusões que a equipe obteve sobre os resultados do projeto, assim como as dificuldades encontradas e propostas futuras.

1 BASE TEÓRICA

Este capítulo tem como principal objetivo explicar as teorias da psicologia para o aprendizado humano, tomando como exemplo dois pensadores da área, Jean Piaget e Lev Vygotsky. Além de assimilar as ideias apresentadas no contexto de jogos educacionais.

Existem diversos métodos, perspectivas e teorias desenvolvidas no histórico da psicologia, para entender como as pessoas assimilam novas ideias, e fazer com que elas aprendam novos conhecimentos. (MATISUI et al., 2012).

Uma das teorias, a chamada Epistemologia Genética ou Psicogenética, também conhecida como concepção construtivista da formação da inteligência, desenvolvida por Jean Piaget é a de que os seres humanos desde seu nascimento constroem seus conhecimentos de forma própria, ou seja, sozinho (ARGENTO, 2008).

Outra teoria vai de encontro com os pensamentos de Jean Piaget, a corrente pedagógica criada por Lev Vygotsky chamada de socioconstrutivismo (FERRARI, 2008). Essa por sua vez, diz que o aprendizado do ser humano vem de interações sociais com o meio e com as pessoas.

1.1 CONSTRUTIVISMO DE JEAN PIAGET

Nascido em 1896, na cidade de Neuchâtel, Suíça, Jean William Fritz Piaget desde pequeno apresentava características que o destacavam de outras crianças em nível de conhecimento, já aos 11 anos publicou um pequeno artigo científico a respeito de observações que fez a um pássaro albino. Aos seus 18 anos Piaget teve que definir seu futuro para uma profissão, entre filosofia e biologia, escolhendo a formação acadêmica Biológica. Com 20 anos, se formou doutor em Malacologia[2]. Piaget desde sua adolescência desejou criar uma teoria biológica do conhecimento e para isso se voltou para a área da psicologia buscando esse ideal. Jean Piaget foi trabalhar no laboratório de Binet onde lá conseguiu dados importantes para a construção da Teoria da Epistemologia Genética (CAETANO, 2013).

Embora tenha sido conhecido como o grande criador da metodologia de ensino construtivista, Piaget não criou nenhuma nova metodologia pedagógica. Passou a sua vida estudando os processos da inteligência e possui uma vasta obra relacionada a esse processo,

[2] Malacologia: Ramo da biologia dedicado a estudar os moluscos.

sua pesquisa acabou abrindo caminho para que outras pessoas pudessem usar um embasamento de sua teoria para criar metodologias pedagógicas novas.

De acordo com Bock, Furtado e Teixeira (1999), a criança não é um adulto em miniatura. Ao contrário, apresenta características próprias de sua idade. Seguindo este raciocínio, Piaget fala que o desenvolvimento do ser humano passa por períodos, cada um com as suas características, ou seja, quando o período não supre mais as necessidades do indivíduo, ele é passa para o próximo período. Esses períodos descritos por Piaget estão organizados da seguinte maneira (BOCK; FURDATO; TEIXEIRA, 1999):

- 1° período: **Sensório-Motor** (0 a 2 anos)
- 2° período: **Pré-Operatório** (2 a 7 anos)
- 3° período: **Operações Concretas** (7 a 11 ou 12 anos)
- 4° período: **Operações Formais** (11 ou 12 anos em diante)

O primeiro período é onde, para a criança, o mundo é caótico, com objetos que ficam sumindo o tempo todo, o espaço e o tempo sentidos, entre outras características. Portanto as crianças possuem funções limitadas, mas por percepção e movimentos, essas funções começam a evoluir podendo assim ter uma melhor sucção ao se alimentar, ou a melhor movimentação dos seus olhos.

No final desse período, a criança começa a construir e organizar um conhecimento de que o mundo que já não é mais caótico, pois a criança sabe as noções de espaço, tempo, as instabilidades dos objetos que antes ficavam sumindo e a noção de ego começa a aparecer.

No segundo período é onde surge a necessidade da fala que para Bock, Furtado e Teixeira (1999) é a possibilidade de exteriorização da vida interior, podendo esquematizar suas ações e comunicação com eficiência, por outro lado, a capacidade com números e ordem não são tão eficientes, pois a criança além de ainda estar centrada nela mesma, não possui a capacidade de determinar um ponto de início do problema, importando somente o ponto presente do problema. E no seu aspecto afetivo, começa a surgir certos sentimentos por pessoas que estão fora do seu mundo, como professores, animais e outros. No final desse período, a função motora da criança está mais desenvolvida, podendo pegar pequenos objetos, e sua coordenação motora é melhorada, podendo se expressar por meio de desenhos abstratos.

No terceiro período, chamado de operatório-concreta, de acordo com Bock, Furtado e Teixeira (1999), o egocentrismo presente na fase anterior é superado pela construção lógica,

isto é, a capacidade da criança de estabelecer relações que permitam a coordenação de pontos de vistas diferentes. As relações de números e ordem não são mais um problema, pois a criança obteve uma nova ferramenta mental: Operações, ou seja, agora as crianças tem um ponto de partida de um problema e podem voltar, para um melhor raciocínio, já possuem um melhor entendimento sobre comprimento, volume e outra medida de espaço. Outra característica é a cooperação, como a criança já consegue ver alguma coisa sobre outro ponto de vista, facilita a questão do trabalho em grupo, organização de ideias, tomada de decisão em grupo, entre outras atividades.

O último período é o operatório-formal, que segundo Bock, Furtado e Teixeira (1999), a criança passa a ser um adolescente, e passa a construir seus pensamentos no plano da ideia fazendo com que tenha uma visão mais crítica e passe a desenvolver suas próprias teorias, principalmente sobre o que ele desejaria mudar na sociedade, sem precisar das referências reais que ele tinha como no caso do período anterior. Vendo pelo lado das relações em grupo, o jovem tem como referencial principalmente o seu grupo de amizades, tornando a sua moral igual a moral do grupo.

Como se pode ver a visão construtivista de Piaget, definiu distintamente as fases da inteligência e os pensamentos construídos por cada um e se percebe também as mudanças de um período para o outro, vai de acordo com o meio onde a criança esteja, ou seja, a inteligência nesse caso não vem herdada geneticamente, mas sim de interações com o meio físico.

1.2 CORRENTE SOCIOCONSTRUTIVISTAS DE VYGOTSKY

Vygotsky foi um professor e estudioso de literatura e psicologia, nascido em 1896 em Orsha, Bielo-Rússia. Vygotsky começou sua pesquisa literária em 1917 após graduar-se na Universidade de Moscou com especialização em literatura. De 1917 a 1923 deu aulas de literatura e psicologia em uma escola de Gomel, Bielo-Rússia. Em 1924 se mudou para Moscou onde trabalhou no instituto de psicologia e também no instituto de Estudos das Deficiências (VYGOTSKY et al., 1991).

Vygotsky reuniu entre 1925 e 1934 muitos grupos de jovens cientistas das áreas da psicologia e no estudo das anormalidades mentais. Em 11 de junho de 1934 faleceu em decorrência da tuberculose. (VYGOTSKY et al., 1991).

Apesar do curto período de vida Lev Semenovitch Vygotsky foi um grande pensador e seu trabalho deu origem a corrente socioconstrutivista (FERRARI, 2008).

Vygotsky acreditava que o conhecimento e a aprendizagem são passados através de interações sociais, mas principalmente através da discussão o ato da fala era o principal meio para se passar experiências aos mais novos (VYGOTSKI; BRUNER; MINICK, 1998). Para isso ele argumentou que "as formas mais elevadas de comportamento humano tem sua origem na interação social, e a participação social no comportamento social é mediado pela fala" (VYGOTSKI; BRUNER; MINICK, 1998, p. 21, tradução nossa).

Segundo Filho (2008), Vygotsky tinha um enfoque maior no processo de aprendizagem propriamente dito do que no desenvolvimento maturacional do conhecimento, isso significa na prática que o desenvolvimento cognitivo para Vygotsky vinha **de fora para dentro** do objeto, e não de **dentro para fora** através da interação do objeto com o meio, pensamento esse tido por Piaget.

Vygotsky não desenvolveu uma concepção estruturada para nomear as fases do desenvolvimento humano separadas pela idade da pessoa e sua maturidade mental como Piaget (FILHO, 2008). Porém para que se tenha um melhor entendimento dos pensamentos de Vygotsky é necessário falar de conceitos por ele criados. São eles (ROSA et al., 2012):

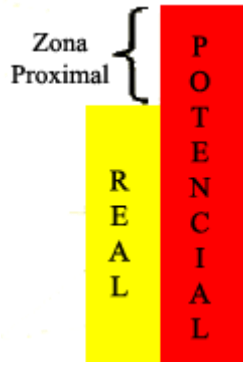
- **Zona de desenvolvimento real.**
- **Zona de desenvolvimento potencial.**
- **Zona de desenvolvimento proximal.**

Segundo Rosa et al. (2012) a chamada zona de desenvolvimento real, é o conceito que diz que um indivíduo já alcançou todas as habilidades e conhecimentos necessários para solucionar um problema, sem precisar de ajuda de agentes externos, como outras pessoas, ou mesmo livros.

Outro conceito desenvolvido por Vygotsky foi a chamada zona de desenvolvimento potencial que é o conjunto total de conhecimentos necessários para que o indivíduo consiga solucionar seus problemas (ROSA et al., 2012).

Por fim, o último conceito é a chamada zona de desenvolvimento proximal. Para Filho (2008), é o caminho que o ser humano tem que seguir para conseguir desenvolver funções que estão em estado de maturação com o objetivo de que se tornem funções já consolidadas (zona de desenvolvimento real).

Figura 1: Zona de Desenvolvimento Proximal



Fonte: Adaptado de Rosa et al. (2012, p. 2)

Pode-se observar com esses conceitos criados por Vygotsky que "Aprendizagem e desenvolvimento [...] relacionar-se-iam de maneira circular. É possível concluir, assim, que se desenvolve mais quem melhor aprende e quanto mais se aprende mais facilitado torna-se o processo de desenvolvimento." (FILHO, 2008, p.273) sendo necessária a ação do aprendizado vindo do meio externo para que o indivíduo consiga chegar ao estado de conhecimento pleno, ou seja, a zona de conhecimento real.

Portanto pode-se concluir que as vertentes construtivistas e socioconstrutivistas criadas por Piaget e Vygotsky respectivamente, não são antagônicas, mas apenas diferentes no sentido de qual foco elas tomam como o mais importante para a criação do conhecimento humano, sendo que uma não exclui a outra, ou seja, da mesma forma que Vygotsky vê a interação com o meio um importante ponto para se atingir um estado de conhecimento maior, Piaget não discrimina o fato das interações sociais ajudarem na formação do indivíduo (FILHO, 2008).

1.3 JOGOS EDUCACIONAIS

Os jogos ao longo da história já vinham sendo utilizados para passar conhecimento, como por exemplo através de um **jogos de guerra**, introduzidos por volta de 1780 com Helwig e Lieutenant Von Reisswitz (SIMON, 2005). No século 19, o exército alemão usou o jogo *Kriegsspiel*, criado por Helwig e aprimorado por Reisswitz, como uma maneira de simular o campo de batalha. Em 1824, o jogo já era recomendado para o treinamento de oficiais e cópias dele eram distribuídas para todos os regimentos do exército alemão (SIMON, 2005).

Segundo Simon (2005), existem três gerações de jogos com conexão ao aspecto educacional, que são:

- **Primeira Geração**: Possui uma visão mais behaviorista e se baseia no fato de que um conhecimento só é adquirido através da repetição.
- **Segunda Geração**: Tem um enfoque maior na relação entre o jogador e o jogo ao invés da repetição mecânica, o jogo já oferece graus variados de dificuldade conforme a evolução do jogador.
- **Terceira Geração**: Esta geração não se foca apenas no jogo e no jogador em si como uma única coisa, mas sim em todo o processo que envolve o aprendizado, o papel do professor é mais de facilitador do processo do que um instrutor em si, o que lembra um conceito muito utilizado pelos pedagogos adeptos da teoria construtivista de Jean Piaget.

A terceira geração de jogos educacionais defendida por Simon (2005) apresenta claramente a perspectiva de que jogos podem por si próprios proporcionar o conhecimento desejado a seus jogadores.

Para entender melhor como jogos podem passar conhecimento, basta entendermos melhor nossa própria capacidade de memorização.

> Dos três componentes que constituem a memória de trabalho, o mais importante é o executivo central, ou sistema de controle de atenção. [...] O executivo central é o principal componente da teoria, e tem como funções o raciocínio, a tomada de decisões, o planejamento de estratégias e o controle do comportamento por meio da integração das informações dos sistemas subordinados. Sua operação envolve a aprendizagem e aplicação de regras contingentes, raciocínio abstrato, e manutenção da atenção e concentração e seu funcionamento são estreitamente relacionados às áreas pré-frontais. (MESSINA; TIEDEMANN, 2009, p. 214).

Messina e Tiedemann (2009) também evidenciarão, enquanto estudavam o desempenho de crianças com déficit de atenção, que todas as atividades internas a uma mesma área do cérebro possuem uma forte relação entre elas. Isso implica em dizer que através da **atenção**, **concentração** e **raciocínio abstrato**, um raciocínio coincidentemente muito utilizado em programação, é possível de se chegar ao aprendizado.

Mesmo tendo diferentes pontos de vista, na prática, ambas as idéias podem ser identificadas em um vídeo da *Valve Corporation* (2011). A dinâmica apresentada consistia em trazer aluno de 7ª série da escola *Evergreen – Shoreline, Washington* para o estúdio de desenvolvimento de jogos da empresa. Lá os alunos fizeram uma atividade para entenderem como os conteúdos de matemática e física ensinados em sala de aula eram utilizados em jogos e na vida real.

A ferramenta utilizada era um software de *level design* do jogo Portal e com a ajuda de desenvolvedores, os grupos de alunos aprenderam o básico do programa para do desenvolvimento das fases. Até este ponto, a metodologia se assemelha às teorias propostas por Vygotsky onde o conhecimento é passado por um instrutor. Logo após a execução das primeiras fases criadas, o desempenho dos alunos aumentou bastante, causando até espanto nos próprios idealizadores da atividade (VALVE CORPORATION, 2011).

O interesse em aprender novas funções presentes no software e em desenvolver fases cada vez mais criativas levou os alunos a um nível diferente de aprendizado, similar ao que Piaget descrevia em seus trabalhos.

Cada um teve seu papel importante para que o estudante aprendesse a desenvolver fases para o jogo. O estímulo do instrutor de Vygotsky levou à proposta do conhecimento, sem isso, o aluno provavelmente nunca teria buscado por algo parecido por falta de conhecimento da ferramenta e interesse na dinâmica. Depois do estímulo, a experiência de ter criado uma fase levou o jogador rumo ao que Piaget descrevia como a busca de conhecimento por interesse próprio e por esse acontecimento os alunos se prenderam a tal conhecimento e alcançaram resultados extraordinários.

A abordagem a ser trabalhada no jogo Virtual Justice será construtivista, ou seja, o computador ou software vai dar apenas as ferramentas necessárias para que o aluno-jogador consiga aprender a seu modo.

A máquina será um elemento instigador com a função de proporcionar o desenvolvimento do jogador, e fazendo-o ter curiosidade e o estimulando a ponto de que consiga resolver os desafios propostos (TAKASHIMA, 2004).

Tendo como ideia principal instigar o jogador, a ver os novos inimigos, ou que outros cenários ele pode encontrar com novos desafios, um jogo com abordagem no construtivismo de Piaget seria uma boa forma de fazer com que o aluno não perca o interesse no que está sendo-lhe ensinado.

Tendo como referência a teoria de Vygotsky, o jogador, por meio de interações com a protagonista do jogo, terá uma espécie de relação social, isso amadurecerá certas habilidades em seu caminho no jogo, fazendo com que o objetivo do jogo seja alcançado, ou seja, o jogador terá todas as ferramentas para resolver o problema. Sendo assim o jogo serve como uma zona de desenvolvimento proximal, levando o jogador a chegar, utilizando essas ferramentas, a uma zona de conhecimento real.

Também remetendo ao construtivismo de Piaget, no que se diz aos períodos da inteligência, um jogador que não tenha muito conhecimento na área, poderia passar pelas

mesmas experiências de um indivíduo do primeiro período, se familiarizando com as mecânicas de jogo, construindo um conhecimento, até chegar ao momento de abstração, no caso se referindo ao último período.

2 CONCEPÇÃO DA IDEIA

Tendo conhecimento dos fundamentos teóricos para a caracterização e construção do construtivismo, é possível de se elaborar caminhos para alcançá-lo. Estes deverão ser bem definidos para que o processo de concepção da ideia não seja equivocado.

“Os jogos digitais podem ser definidos como ambientes atraentes e interativos que capturam a atenção do jogador ao oferecer desafios que exigem níveis crescentes de destreza e habilidades (BALASUBRAMANIAN; WILSON, 2006)” (SAVI; ULBRICH, 2008, p. 2).

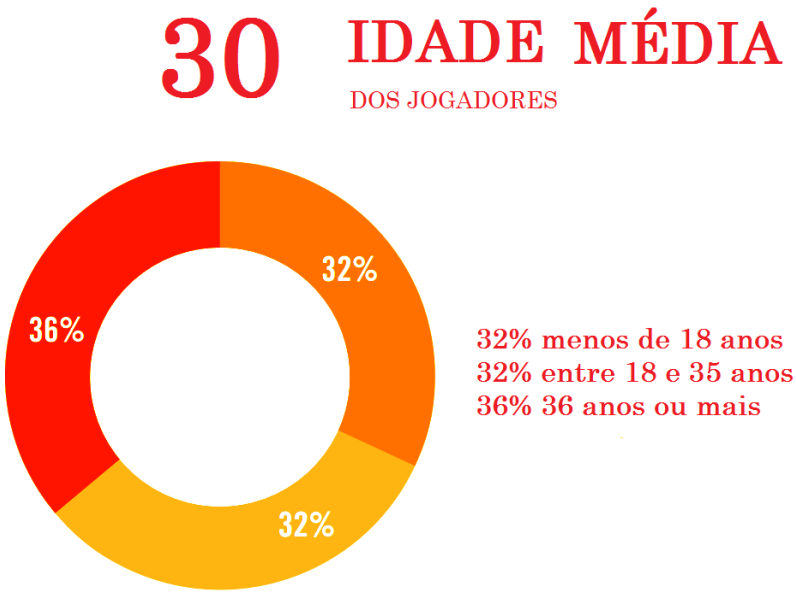
O Design de jogos é exatamente a ideia produzida, com o objetivo de passar uma experiência para quem o joga e desencadear o prazer em jogar (SCHELL, 2011) consequentemente prendendo o jogador na atividade de maneira voluntária. Schell (2011) também menciona que cada pessoa possui uma experiência diferente sobre a mesma coisa, isso se dá pelo fato delas possuírem personalidades e áreas de interesse individuais.

O primeiro passo para se definir do jogo que será desenvolvido é a escolha do público-alvo e que tipo de experiência desejamos que eles possam ter durante o jogo.

2.1 PÚBLICO-ALVO

O público-alvo, no caso, é o próprio jogador e, ao mesmo tempo, o indivíduo que receberá o **conhecimento** de programação básica e orientada a objetos. A partir do **conhecimento**, podemos definir um intervalo de idade ideal para o aprendizado de programação.

Figura 2: Faixa Etária de Jogadores de *Videogame* de 2013 nos Estados Unidos



Fonte: Adaptado de Entertainment Software Association (2013, p. 2).

Conforme o gráfico acima mostra, se as duas primeiras partes forem juntadas podemos afirmar que 64% do público que joga videogame possuem até 35 anos.

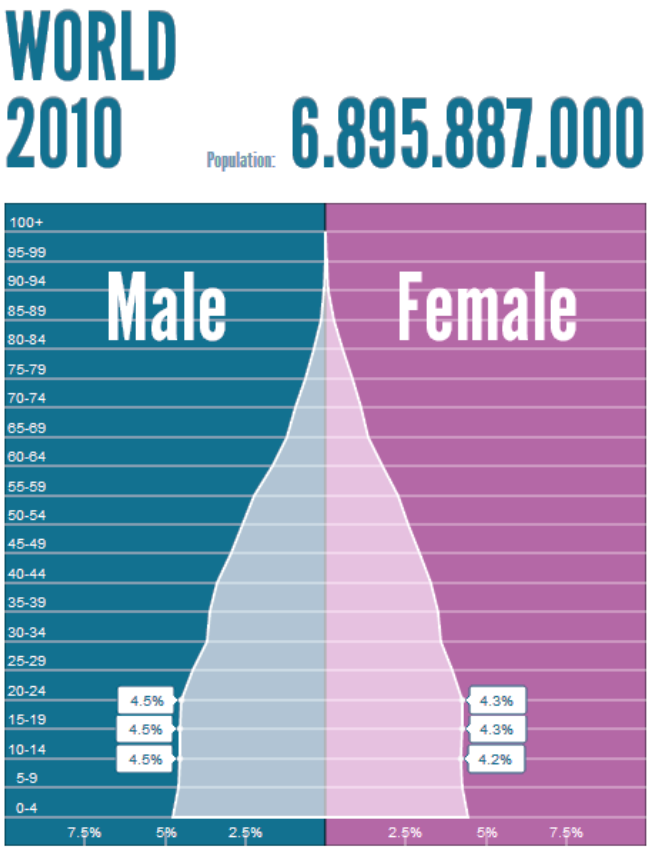
O motivo para termos escolhido esta parte da população é devido a grande quantidade de público que o jogo poderá alcançar além de ser o público que se encontra normalmente no espaço acadêmico e busca se qualificar para o mercado de trabalho.

Ainda assim a quantidade de público ainda é muito grande e diversificada, precisa-se limitar ainda mais o público para que ele consiga atingir um jogador mais específico. Tendo como base a reportagem da VentureBeat, alunos de 9 anos já aprendem programação e, segundo notícia publicada no Estadão (2011, online), a idade média dos alunos estudantes de ensino superior é de 26 anos. Esse intervalo de idade de 9 a 26 anos é o mais apropriado para o jogo, no entanto, ao longo do desenvolvimento da ideia outros limites podem levar ao aumento da faixa etária.

De 10 a 24 anos existem 3 perfis de jogadores: pré-adolescente, adolescente e adulto jovem. Esse público, independente de ser masculino ou feminino, se divide em três momentos da vida a começar por uma fase de constantes questionamentos e grande capacidade neurológica, passando pelo preparo para a vida adulta de formando opiniões e terminando no estágio em que passam a se dedicar mais ao trabalho que aos jogos (SCHELL, 2011). Isso

demonstra que se o projeto atingir exatamente esses três públicos o jogo tem a possibilidade de se tornar uma grande ponte entre a fase acadêmica da vida e o mercado de trabalho.

Figura 3: Pirâmide Etária Mundial de 2010



Fonte: Adaptado de UNITED NATIONS (2011, online).

A pirâmide etária mundial acima é de 2010 aponta que 26,3% de uma população de aproximadamente sete bilhões de pessoas possuem um perfil próximo do esperado, mas ainda faltam muitos fatores a serem considerados e que reduzem a quantidade de jogadores, entretanto, mesmo que fosse somente 1% da população contida no intervalo do público-alvo, o número de pessoas ainda assim continua grande o suficiente para se considerado um projeto que possa contribuir bastante com o modo como o mundo enxerga os jogos educacionais.

2.2 OS 5 GRANDES FATORES

Tudo que um bom design de jogo busca alcançar é o interesse do jogador. Fazer com que o jogador permaneça imerso no mundo e busque a diversão em jogar. Jogos educacionais acima de tudo são jogos e por essa razão devem buscar sempre em primeiro lugar o prazer, caso o contrário estará correndo o risco de se tornar uma atividade chata e desinteressante.

VandenBerghe (2011) propõe adaptarmos uma outra metodologia da psicologia para identificarmos o perfil do nosso público-alvo chamada O.C.E.A.N. que adaptado para o Brasil podemos chamar de **Os 5 Grandes Fatores**. (HUTZ, 1998)

Os 5 Grandes Fatores, segundo VandenBerghe (2011), são características psicológicas que definem as motivações que cada indivíduo possui. Eles são:

- ***O**penness to Experience* (Abertura para a experiência)
- ***C**onscientiousness* (Escrupulosidade)
- ***E**xtroversion* (Extroversão)
- ***A**greeableness* (Sociabilidade)
- ***N**euroticism* (Neuroticismo)

Cada fator define se um indivíduo possui grandes ou pequenos traços do mesmo, sendo possível que o indivíduo gere fatores contrários aos citados. Cada um também possui 6 cargas fatoriais que detalham ainda mais a personalidade, totalizando 30 cargas fatoriais. (HUTZ, 1998)

VandenBerghe acaba então criando novas definições adaptadas para o game design que o permitem definir melhor o gosto de um jogador partir de tal metodologia. Ele também explica que o objetivo é tentar englobar o maior número de pessoas possível contanto que não comprometa a lógica do design.

Figura 4: Exemplificação da motivação de *Assassin's Creed 2*

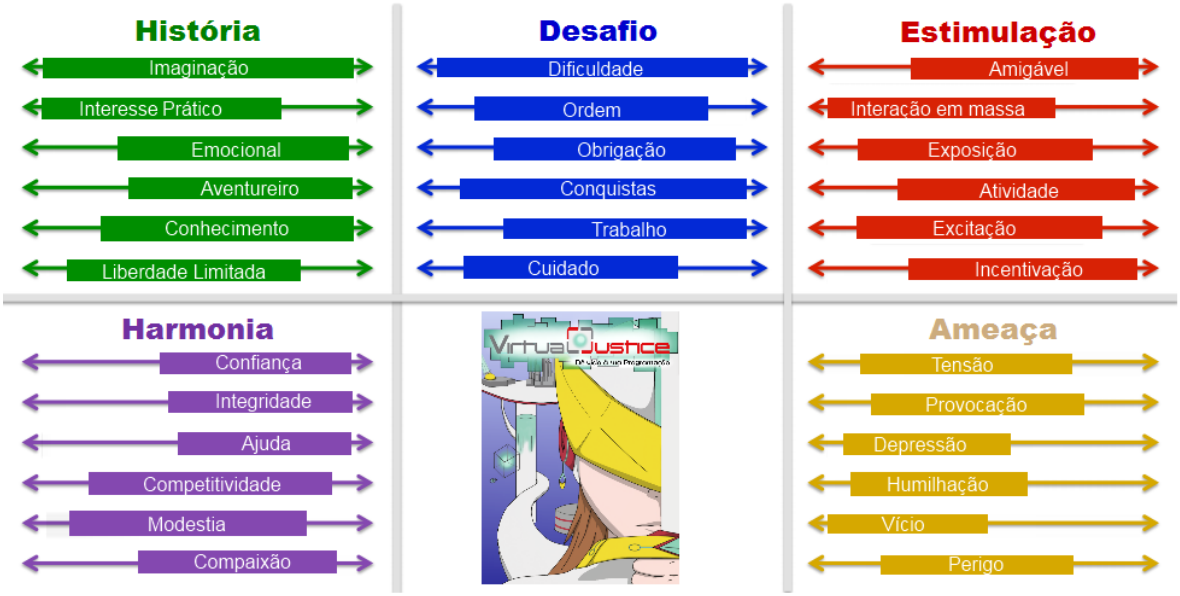


Fonte: The 5 Domains of Play (2012, p. 70)

Na Figura 4, podemos observar que o jogo *Assassin's Creed 2*, por exemplo, foi avaliado segundo os 5 grandes fatores apresentando de forma detalhada as características do projeto.

Incorporando a metodologia de VandenBerghe em nosso trabalho, como mostra a Figura 5, é possível evidenciar as características de cada fator não apresentados na figura anterior.

Figura 5: 5 Grandes Fatores aplicados ao Virtual Justice



Fonte: Autores (2013)

No que se refere ao fator história, ele descreve o jogo indicando se ele possui ou não conteúdo externo as ações do indivíduo, basicamente o trabalho desenvolvido pelos produtores do jogo.

O fator desafio remete bastante aos questionamentos de Schell (2011) sobre que tipos de dificuldades e realizações o jogador terá ou gostará de fazer durante o tempo em que estiver jogando.

Estimulação é um fator bastante relacionado às interações do jogador com os personagens que são controlados, sendo possível encontrar tanto jogadores que busquem similaridades com seus avatares como aqueles que escapam do cotidiano para vivenciar exatamente aquilo que não podem ser fora dos *videogames*.

Para caracterizar o jogo no que se refere às habilidades de interação social do indivíduo com os outros elementos do jogo e até mesmo com jogadores em modo *multiplayer*, o fator harmonia procura destacar o perfil da maioria dos indivíduos com que interagimos.

Por fim, o fator ameaça procura descrever, dependendo do quão envolvente o jogo for, a sensação que o mesmo desperta nos indivíduos ao longo da experiência.

O trabalho espera alcançar indivíduos que sejam abertos a novas experiências. Focar na história e ambientação será o principal fator a ser explorado no jogo, fora isso os outros fatores seriam bem balanceados. Se novas experiências são o que levariam o público a comprar o jogo, então a primeira coisa a se desenvolver é a experiência que desejamos passar.

2.3 EXPERIÊNCIA DE JOGO

Coincidentemente Schell (2011) também afirma que o primeiro passo no desenvolvimento de um jogo é a definição das experiências que serão passadas. São elas que proporcionam o prazer em jogar, e se caracterizam pela construção de todos os eventos que acontecem no jogo a partir de decisões tomadas pelos jogadores. Uma experiência nunca será a mesma para duas pessoas.

A experiência que pretendemos passar é a de que o jogador deverá programar algo dentro do jogo. Com isso, a ideia é contar a história do ponto de vista de um programa de computador.

Decidiu-se que, para agradar também ao público feminino, o jogo deve possuir características tais como uma personagem que tenha traços femininos, mas que ao mesmo tempo demonstre coragem, força e bastante carisma. Em compensação o jogo pretende também agradar ao público masculino com elementos de jogos de ação e aventura, muitas lutas e poderes para serem melhorados. No final, conseguiu-se produzir a ideia de um jogo que atenda a todo o embasamento teórico adquirido e o resultado será levado para o desenvolvimento.

Figura 6: Astra.IA



Fonte: Autores (2013)

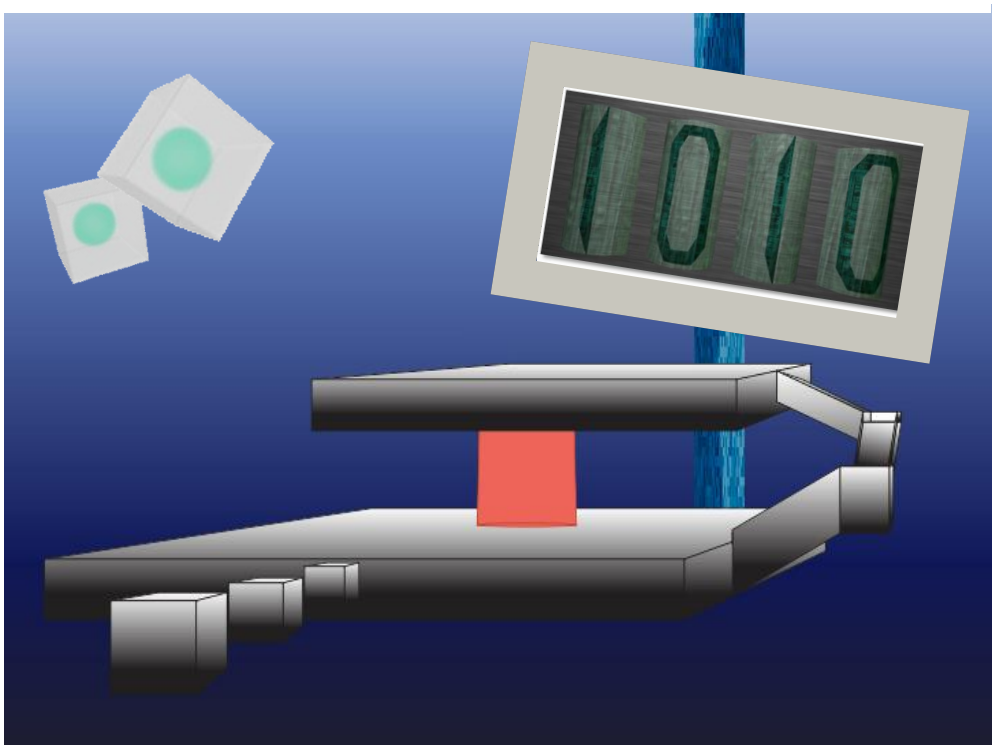
A história se passa dentro do computador. Todos os seres que vivem em paz no ambiente são arquivos e programas que trabalham constantemente para promover a informação desejada ao usuário. Um dos programas, mais especificamente um antivírus *Open Source* chamado **Astra.IA**, patrulha a procura de vírus e arquivos que tenham atividades suspeitas para poder apreende-los e tomar suas devidas precauções para que eles não voltem a incomodar.

Astra acaba caindo na armadilha de um novo vírus muito perigoso, o **C4OS**. Ele possui habilidades que acabam a deixando-a indefesa. Como única alternativa, ela reinicia o sistema em modo de segurança e é transferida para o ambiente mais primitivo do computador, a BIOS. Astra terá que contar a partir daí com a ajuda do jogador para recuperar seus códigos de defesa, derrotar todos os perigos que encontrar pela frente e salvar o computador do controle absoluto que o vilão tentará conquistar.

Durante o jogo, Astra construirá uma amizade com o próprio jogador que será o responsável por sua recodificação. Ela terá que também ensinar ao avançar do jogo conceitos e lógicas de programação para que o programador entenda o que ela pretende ensinar e praticar seus conhecimentos na própria personagem. A evolução no jogo se dá pela criação de novos objetos que servirão como armas e equipamentos e a modificação de atributos que definem as habilidades de jogo da personagem.

Para balancear o aprimoramento da personagem durante a história, foi criado o ***Pixel Point***. Ele é um elemento do jogo que representa a informação presente em todos os personagens virtuais, tanto que ao derrotar um inimigo Astra conseguiria absorver essas informações e assim possibilitar que o programador codifique ainda mais.

Figura 7: Representação dos elementos presentes no jogo



Fonte: Autores (2013)

É muito importante também que o jogador sinta que esteja dentro do computador. Todas as fases foram idealizadas para que representassem vários componentes presentes na máquina. O próprio avanço da história será uma subida para níveis onde os detalhamentos gráficos e sonoros fossem superior aos anteriores demonstrando o contato mais próximo que as fases têm com o jogador.

Todas essas ideias foram colocadas em prática para a construção do produto. Entretanto, foi preciso a utilização de programas que gerassem os elementos e ações que estarão contidas no jogo.

3 TECNOLOGIAS UTILIZADAS

Para transformar a ideia em aplicação faz-se necessário a utilização de ferramentas capazes de produzir cada um dos elementos presentes no jogo.

Os arquivos de áudio foram produzidos em ambiente diferente da produção do jogo sendo um dos casos a gravação da voz em um estúdio com isolamento acústico e máquinas com placas de áudio que trabalham com 12 canais, efeitos sonoros foram baixados gratuitamente e as músicas de fundo foram produzidas por músicos em comum acordo com o projeto.

3.1 ASTAH COMMUNITY

Astah *Community* foi um programa importante utilizado para a concepção do projeto. Capaz de produzir diagramas UML para organizar e estruturar o modelo do software, o aplicativo é bastante utilizado em engenharia de software para apresentação simplificada de cada funcionalidade presente no programa.

A *Change Vision*, produtora do software em questão, disponibiliza as versões mais atualizadas do Astah *Community*, e sua compatibilidade com versões anteriores facilita o acesso a diagramas produzidos por trabalhos acadêmicos antigos que tenham salvado em arquivos **.asta**.

3.2 UNITY 3D

O programa mais utilizado pela equipe no processo de desenvolvimento do jogo foi o *Unity* 3D, um motor de jogo que ajuda a produzir jogos multiplataforma de maneira mais dinâmica e simples. O motivo para ter escolhido este motor foi por possuir uma interface fácil de compreender e ter mais qualidade que outros que permitem a utilização gratuita do programa.

Unity Technologies é a companhia criadora do motor e detentora de uma comunidade de 1,5 milhão de desenvolvedores incluindo grandes distribuidoras de jogos, estúdios independentes, estudantes e amadores. Para permanecer na frente em inovação, ela investe incansavelmente na ferramenta de desenvolvimento 3D que é ganhadora de prêmios, o

próprio *Unity*, e iniciativas de democratização do produto e serviços prestados pela companhia (MARKET WATCH, 2013).

A versão utilizada permaneceu a mesma desde o começo do projeto. Os scripts para a programação de jogos são escritos em Javascript, C# e Boo, possui integrado a ele duas tecnologias auxiliares, a biblioteca de códigos de física da PhysX, versão atualizada de 2011, e um *Integrated Development Environment* (IDE) chamado *MonoDevelop* que será descrito mais adiante.

3.3 DETONATOR

Detonator é uma extensão para o *Unity* 3D, um conjunto de códigos e objetos voltados apenas para criar explosões mais reais dentro do ambiente da *engine unity*, criado por Ben Throop em 2009, e aperfeiçoado desde então (THROOP, 2009). A versão utilizada no projeto é a 1.3b.

3.4 BLENDER

Para a criação dos elementos do jogo como personagens, itens e cenários, foi necessário a utilização do *Blender*, um programa completo para a qualquer atividade que necessite modelar, animar e simular mundos tridimensionais (BLENDER, 2013).

Por ser *Open Source*, o *Blender* é a escolha mais indicada para a produção acadêmica por ser gratuita, mas também permitir a produção de algoritmos em *python* para renderização[3], modelagem, texturização e animação de objetos assim como simulação de forças físicas.

3.5 IDE'S

Os ambientes de desenvolvimento utilizados durante o projeto foram bem importantes para a construção de todo o jogo. Foram utilizadas duas IDE's em especial: *Visual Studio* e *MonoDevelop*. Ambos foram utilizados por trabalharem com o desenvolvimento de códigos em C# que é a linguagem principal do jogo.

[3] Renderização é um processo para se obter um produto final seja ele 2D(Sons) ou 3D (Modelos 3D)

3.5.1 Microsoft Visual Studio 2010 Express

Microsoft Visual Studio é uma IDE desenvolvida pela Microsoft especialmente para programação de códigos da plataforma .NET (MICROSOFT, 2013). Por isso, ela possui ótimas ferramentas e funções que agilizaram tanto no desenvolvimento da Virtual Justice *Language*(VJL) como na programação dos scripts em C# que o *Unity* usa no jogo. Existem duas versões disponíveis de serem usadas, a versão *Express* que é limitada em casos diferentes deste projeto e a versão 2011 adquirida para estudantes pelo programa *Student to Bussiness* da Microsoft em parceria com a instituição de ensino Centro Universitário do Estado do Pará (CESUPA).

3.5.2 MonoDevelop

Integrado ao *Unity* 3.4 existe uma IDE auxiliar, *MonoDevelop* (UNITY TECHNOLOGIES, 2012). A IDE é bastante simples, por isso, boa parte dos códigos tiveram a possibilidade de serem feitos em JavaScript, o que deixa rápido e fácil processamento das informações contidas no jogo.

Atualmente na internet o *MonoDevelop* é bem difícil de ser encontrado, o que durante o desenvolvimento acabou se tornando uma perda considerável de tempo. Como opção alternativa, outros ambientes de desenvolvimento, se instalados corretamente e possuírem compatibilidade com as linguagens de *script*, podem ser sincronizados ao *Unity* 3D para trabalhar em substituição ao *MonoDevelop* assim como foi o caso do *Visual Studio*.

3.6 SOUND FORGE PRO 10

Sound Forge Pro 10 é um dos produtos pagos da Sony para a gravação e edição de material sonoro, no caso das vozes dos personagens que estão presentes no jogo. No caso especial deste produto, o CESUPA cedeu o estúdio de gravação com o auxílio de um profissional da área para as gravações.

3.7 EDITORES GRÁFICOS

As ferramentas de edição gráfica utilizadas foram: *CorelDRAW Graphics Suite X6*, *Adobe Photoshop CS5* e *Paint.NET*. Cada uma foi de grande importância para a geração de

conteúdos visuais de apenas duas dimensões como, artes conceituais, capas e texturização dos objetos do jogo.

Adobe e Corel são empresas que comercializam programas de computação gráfica a mais de 20 anos. Ambas possuem programas de apoio a estudantes e cursos gratuitos por tempo limitado, oferecem parte dos seus pacotes para o auxílio de estudantes que pretendem entrar no mercado de design e produção gráfica (ADOBE SYSTEMS SOFTWARE, 2013) e (COREL CORPORATION, 2013).

3.7.1 CorelDRAW X6

O *CorelDRAW* é um programa de criação e edição de objetos vetorizados[4] incluso no pacote de programas *Graphic Suite X6* e nele é possível a criação de imagens com grande alinhamento, precisão e resolução muito utilizado para a criação de produtos e logomarcas. Também é possível criar moldes 2D de objetos e a partir da exportação para o *Blender* facilita no processo de desenvolvimento.

Para a edição de imagens rasterizadas[5], foi utilizado do mesmo pacote da Corel o *CorelPHOTOPAINT* que manipula a imagem e suas cores com para a geração de imagens mais detalhadas que os criados pelo *CorelDRAW*.

3.7.2 Adobe Photoshop CS5

O Photoshop é um software de edição gráfica, assim como o *CorelPHOTOPAINT*, ele possui ferramentas capazes de trabalhar com imagens mais detalhadas e produzir resultados surpreendentes.

3.7.3 Paint .NET

O único programa de edição gráfico de fato gratuito foi o *Paint.NET 3.4*. Possui semelhanças com o *Paint* padrão da Microsoft, porém possui atributos que o tornam melhor em muitos aspectos, o suficiente para ser utilizado na geração de artes conceituais e digitalização de desenhos desenvolvidos inicialmente no papel.

---

[4] Vetores, em computação gráfica, é a utilização de cálculos vetoriais no intuito de obter retas, curvas, e áreas para a formação de imagens.

[5] Rasterização é um processo de transformação de uma imagem vetorial em uma imagem *raster* (*Pixel* ou pontos).

4 DESENVOLVIMENTO DO VIRTUAL JUSTICE

4.1 VISÃO

A visão do jogo é proporcionar a experiência que um programador sente ao ver seu código funcionando pela primeira vez, trazendo à tona a vontade de completar suas tarefas e cada vez mais avançar na história do jogo.

Com essa motivação, acreditasse que o jogador terá mais capacidade de absorver as informações que serão necessárias para vencer os desafios que, no caso, são os conhecimentos de linguagem e técnica de programação orientada a objetos.

4.2 VIRTUAL JUSTICE LANGUAGE

Tendo a visão em mente, foi desenvolvida uma linguagem de programação, com sintaxe mais próxima do português, fazendo com que termos como **classe**, **método**, **variável** e **atributo**, fiquem mais bem compreendidos na mente de novatos na área de programação.

A linguagem recebeu o nome de VJL (*Virtual Justice Language*), apesar de a linguagem ser toda em português o seu nome foi deixado em inglês com o objetivo de tornar ela mais inserida no contexto do jogo Virtual Justice, e também como forma de deixar a pronuncia mais atrativa.

A VJL foi feita para seguir o paradigma orientado a objetos, sendo assim é possível instanciar objetos e deles invocar comportamentos.

4.2.1 Interpretador

Segundo Apostila (2001, p.17) o interpretador "é um programa que aceita como entrada um programa escrito em uma linguagem chamada linguagem fonte e executa diretamente as instruções dadas nesta linguagem." sendo assim, um interpretador executa diretamente as instruções como se fosse um tipo de máquina virtual.

Diferentemente de um compilador, que segundo Aho, Sethi e Ullman (1995) se trata de um programa que realiza a tradução da linguagem de programação fonte para nível de

maquina, para apenas depois disso ela ser executada e trazer resultados para determinada solução. Um interpretador dispensa o uso de uma linguagem de baixo nível e executa sua programação em uma máquina virtual.

A partir desses conceitos podemos afirmar que a VJL é uma linguagem de programação interpretada, visto que sua programação foi feita para executar linhas de comando, laços de repetição e outras estruturas em uma máquina virtual.

O interpretador da VJL foi desenvolvido completamente desvinculado do jogo Virtual Justice, de forma que seus diagramas e funcionalidades serão apresentados de forma separada do jogo em si.

4.2.2 Escopo

Para criar o interpretador que daria possibilidade de execução ao código da VJL, foi escolhida como base de desenvolvimento a linguagem de programação C#. O motivo da escolha dessa linguagem para se criar o interpretador foi visando o resultado final da linguagem sendo aplicada no jogo, visto que a *game engine* utilizada (*Unity*) utiliza em muitos de seus scripts, C# como forma de programação para seus objetos, o que tornaria a integração do interpretador com o jogo, uma tarefa muito mais simples. Além desse motivo dispensaria a posterior instalação de qualquer outro tipo de maquina virtual ou compilador para fazer o jogo executar, pois o *.NET Framework* já é utilizado para a execução do game, e o mesmo também já será usado para realizar a interpretação dos comandos da VJL.

4.2.3 Desenvolvimento do Interpretador VJL

A ideia de se desenvolver um jogo que propiciasse o aprendizado à programação surgiu em 2012, para que se concretizasse era necessária a criação de uma linguagem de programação traduzida para a língua nativa de forma a facilitar o aprendizado de conceitos de programação como já foi citado acima neste trabalho. Por conta desse motivo a historia do desenvolvimento da VJL esta fortemente ligada a historia de desenvolvimento do jogo Virtual Justice.

A primeira versão do interpretador de comandos foi feita em Agosto de 2012 usando como base para seu desenvolvimento o Java, visto que a primeira versão do Virtual Justice também seria desenvolvida em Java. O exemplo de código abaixo foi da primeira versão que o interpretador conseguia executar:

Figura 8: Exemplo da primeira versão do código executado pela VJL

```
classe Principal {

        global:Personagem personagem = novo Personagem();
        global:Revolver rev = novo Revolver();

        metodo:nulo main(){
                personagem.forca = calculaMelhorForca();
                personagem.habilidade = 53;
                personagem.arma1 = rev;
        }

        metodo:inteiro calculaMelhorForca(){
                var:inteiro tt = 5;
                se(tt == 5){
                                retorne 30;
                }senao{
                                retorne 20;
                }
        }

}
```

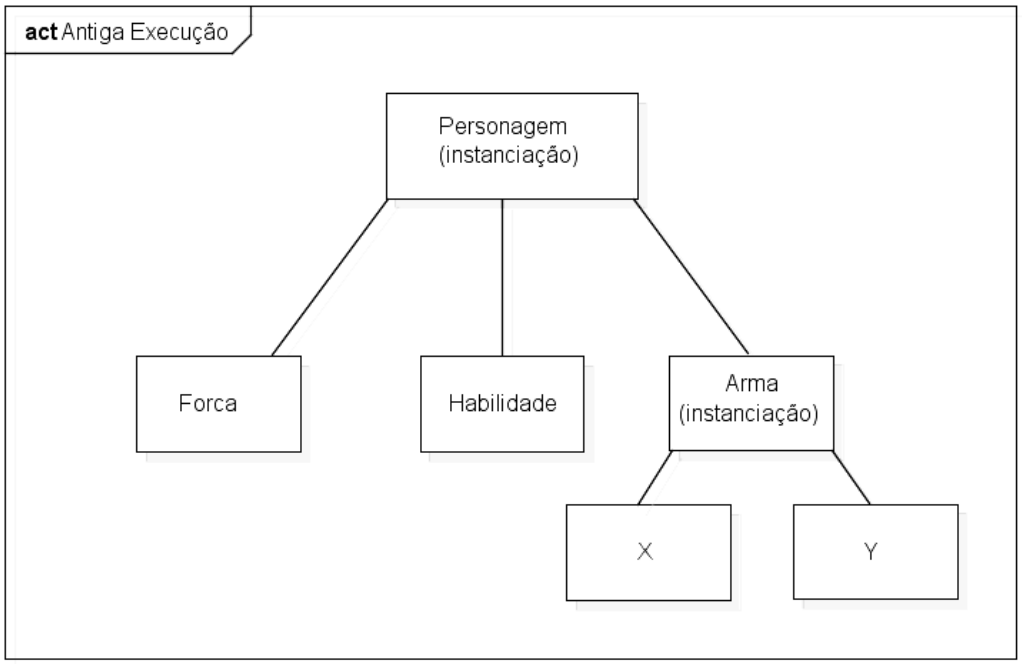
Fonte: Elaborada pelos autores. (2013)

Como pode ser observada na Figura 6 a primeira versão do interpretador deveria ser escrita de modo a compreender e executar uma tipagem forte de variáveis de modo que o programador deveria saber quais eram os seus tipos e, uma vez alocados, não poderiam ser mudados no decorrer da execução do programa por ele feito. O mesmo ocorria com a propriedade de retorno de métodos na VJL, eles deveriam ser criados com ou sem retorno para que o programador saiba também que tipo de resposta seu método irá providenciar.

A primeira versão do interpretador VJL apresentava problemas tanto pela complexidade de um novato na área de se compreender tal carga de conteúdo para se programar na antiga VJL, quanto em seu fluxo de execução, pois nesta versão inicial o interpretador de comandos funcionava com uma estrutura de árvore para a alocação de

memória. Essa estrutura era usada para a declaração de variáveis e a instanciação de objetos, como é demonstrado na Figura 7.

Figura 9: Representação da antiga locação de memoria para a VJL



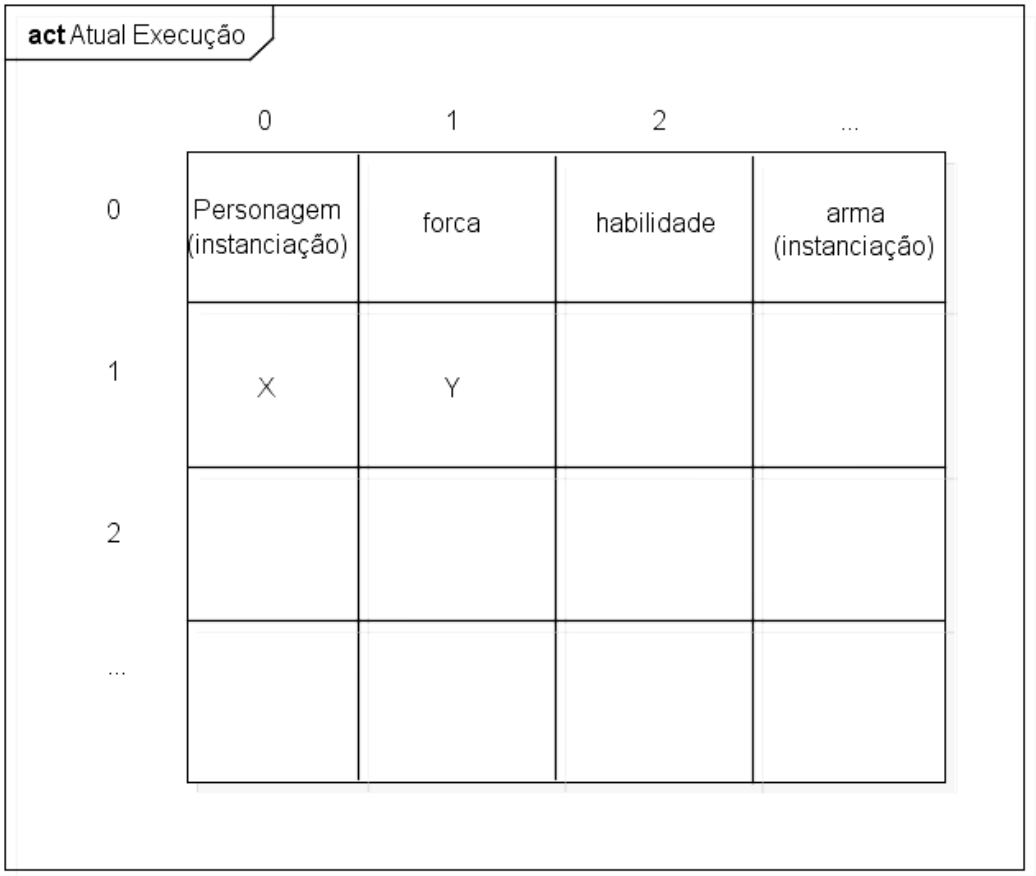
Fonte: Autores (2013)

Essa forma de alocação e controle de memória possuía problemas sérios, aferidos na execução, de forma que não era possível fazer referência a objetos dentro de variáveis, pois, cada vez que uma delas era criada se tornava um novo nodo na árvore de variáveis, sendo assim, uma variável que estava em um nível mais baixo da árvore não conseguia referenciar outra um nível acima.

Para se remediar esse problema o algoritmo realizava uma cópia do atual estado do objeto no nível superior e criava um novo nodo abaixo, porém está solução não resolveu o problema de indexação, visto que, modificações feitas no nível superior não eram replicadas aos “filhos copiados” gerando inconsistência nos estados dos objetos.

Por conta dessa dificuldade vista no desenvolvimento, foi escolhida outra forma para a alocação de memória no atual trabalho, uma que opere de maneira mais fiel aos estados dos objetos e ao funcionamento real da memória RAM: A **alocação por matriz**.

Figura 10: Representação da atual locação de memória para a VJL



Fonte: Autores(2013)

Trabalhando com um sistema de endereçamento de matriz foi possível não só resolver o problema dos estados inconsistentes dos objetos como também criar uma forma mais similar ao de uma memória RAM, simulando um sistema de endereçamento, o interpretador consegue guardar apenas o endereço de localização da instância do objeto e quando a mesma é acessada ou copiada para outra variável no tempo de execução, o que vai ser modificado será a referência, ou seja, o endereço de localização da variável.

Quando a linguagem foi passada para o C# ela já possuía o tipo de implementação por matriz. Além da forma de endereçar variáveis, a VJL passou por uma simplificação em sua sintaxe e também perdeu o padrão de tipagem forte, como mostra a Figura 9.

Figura 11: Atual sintaxe da linguagem VJL

```
classe Personagem{

        atributo forca = 100;
        atributo nome = "Astra";
        atributo itens = nova ListaEncadeada();

        construtor Personagem(){
                variavel x = calculaMelhorForca();
        }

        metodo calculaMelhorForca(){
                variavel s = 10;
                se(s == 100){
                        retorne 100;
                }senao{
                        retorne 10;
                }
        }

}
```

Fonte: Autores (2013)

Com essas mudanças buscou-se uma melhor compreensão por parte do programador iniciante para com os termos, outros pequenos detalhes foram adicionados, como por exemplo, a possibilidade de se escolher a palavra **novo** ou **nova** para se instanciar um objeto, dependendo do nome da classe que o programador escolher. Outros detalhes e explicações para a linguagem poderão ser vistas no apêndice A deste trabalho.

4.2.4 Diagramas

Está seção do trabalho visa mostrar a partir de artefatos gerados usando UML (*Unified Modeling Language*) 2.0 o planejamento e desenvolvimento do interpretador de códigos VJL.

"UML é uma linguagem para especificação, visualização, construção e documentação de artefatos de sistema de software" (MELO, 2010, p.35).

Segundo Melo (2010) no que tange a especificação, a UML ajuda nas tomadas de decisões que ocorrem no desenvolvimento de sistemas, apresentando através de modelos gráficos descrições bem mais completas e fáceis de serem compreendidas pela equipe de desenvolvimento do sistema. Para atingir seu objetivo os modelos gerados devem ser precisos, completos e sem ambiguidade, deve refletir uma representação gráfica simples, porém sem perder a expressividade, do contrário eles teriam o efeito contrario que foram propostos.

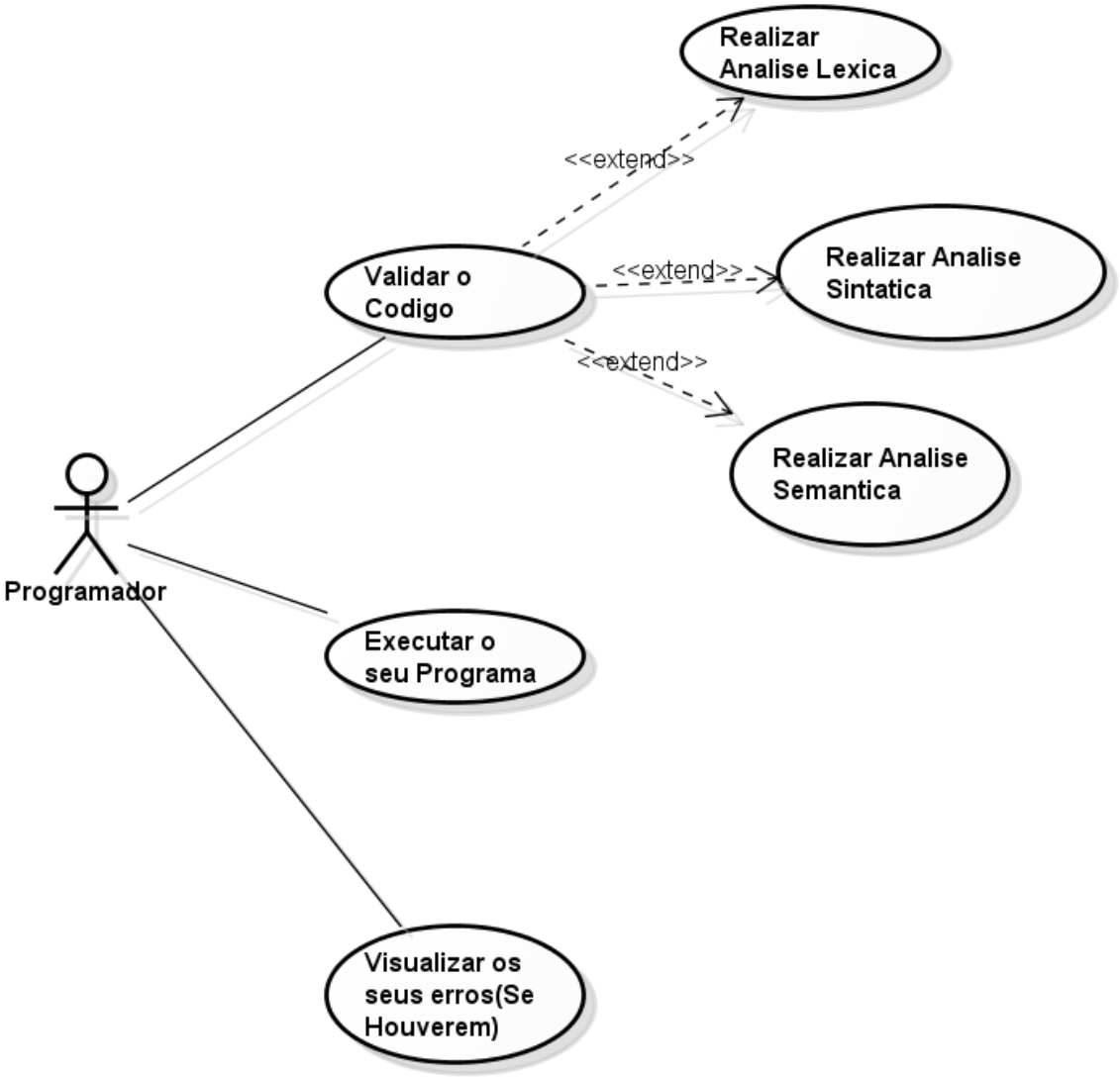
4.2.4.1 Diagramas de Caso de Uso

Segundo Melo (2010) o diagrama de caso de uso é como se fosse o braço direito do desenvolvedor, pois o auxilia a validar requisitos extraídos durante as conversas com o usuário do sistema.

Nesta seção é mostrado o diagrama de caso de uso para o interpretador da linguagem desenvolvida, tendo somente um ator, o programador, ele demonstra tudo o que o mesmo pode fazer com o interpretador.

Para o trabalho também foram gerados detalhamentos de caso de uso para cada funcionalidade, esses documentos podem ser encontrados na seção de apêndices marcados de B a G.

Figura 12: Diagrama de Caso de Uso para o interpretador VJL



Fonte: Autores (2013)

4.2.4.2 Diagramas de Classes

"Um diagrama de classe indica um conjunto de classes, interfaces, colaborações e relações, em geral, de dependência generalização e associação" (RAMOS, 2006, p.97).

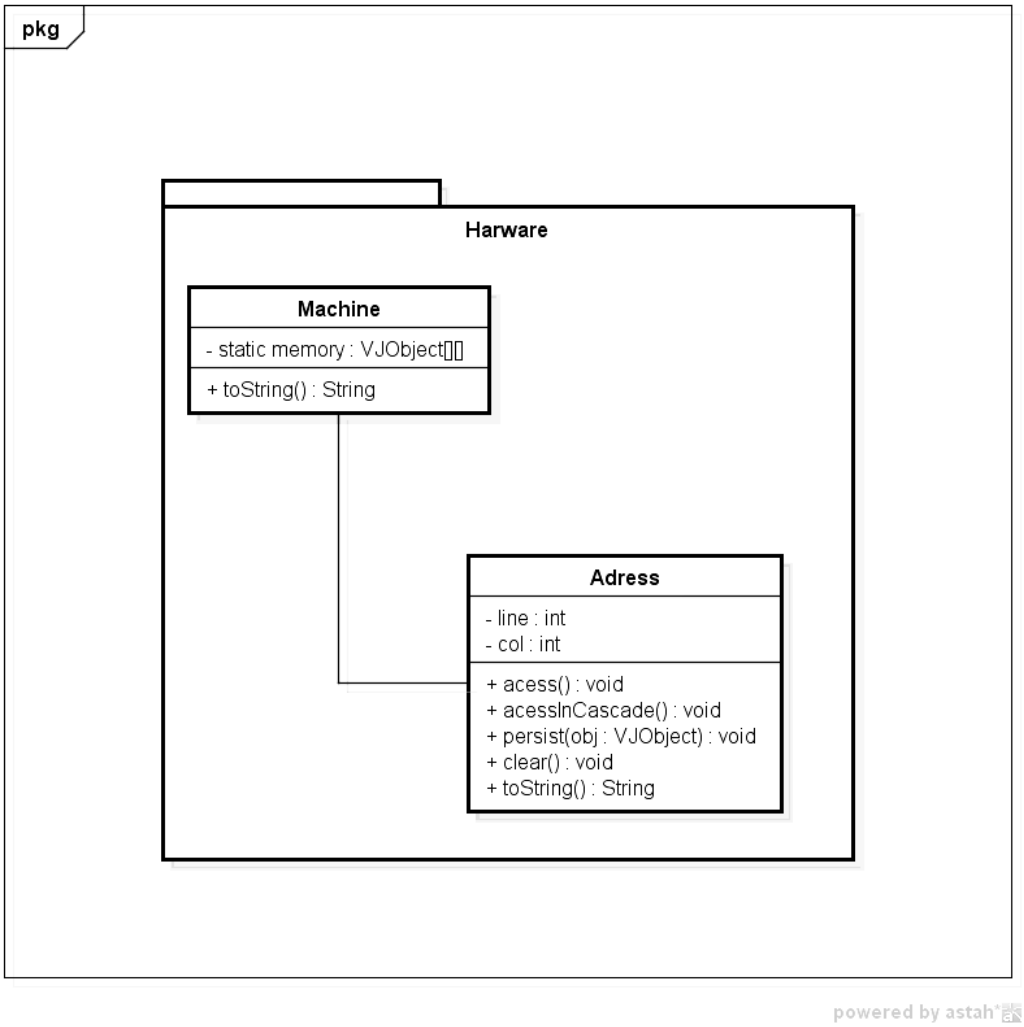
Para uma melhor visão e planejamento do projeto do interpretador VJL seu desenvolvimento foi separado em três partes, gerando assim três diagramas de classe dispostos abaixo, cada um responsável por descrever o funcionamento de ações tomadas por um interpretador de código.

O primeiro diagrama representa as classes usadas para se gerenciar recursos de maquina, espaços na memória RAM.

O diagrama consiste em duas classes a primeira (‘Machine’) é usada para criar uma abstração da maquina e seus espaços de memória, local este usado para armazenar os valores de atributos e métodos criados na VJL, a segunda classe consiste em um controle de

localização de objetos na maquina, ‘Adress’, esses objetos armazenam a linha e a coluna da matriz de RAM em que determinado objeto VJL está guardado.

Figura 13: Classes para gerenciamento de recursos



Fonte: Autores (2013)

Outro diagrama de classe gerado foi o responsável por representar as analises que um interpretador de código deve fazer antes de executar certa linguagem. Este por sua vez o mais extenso, contendo três pacotes (***utils***, ***containers***, ***analizers***).

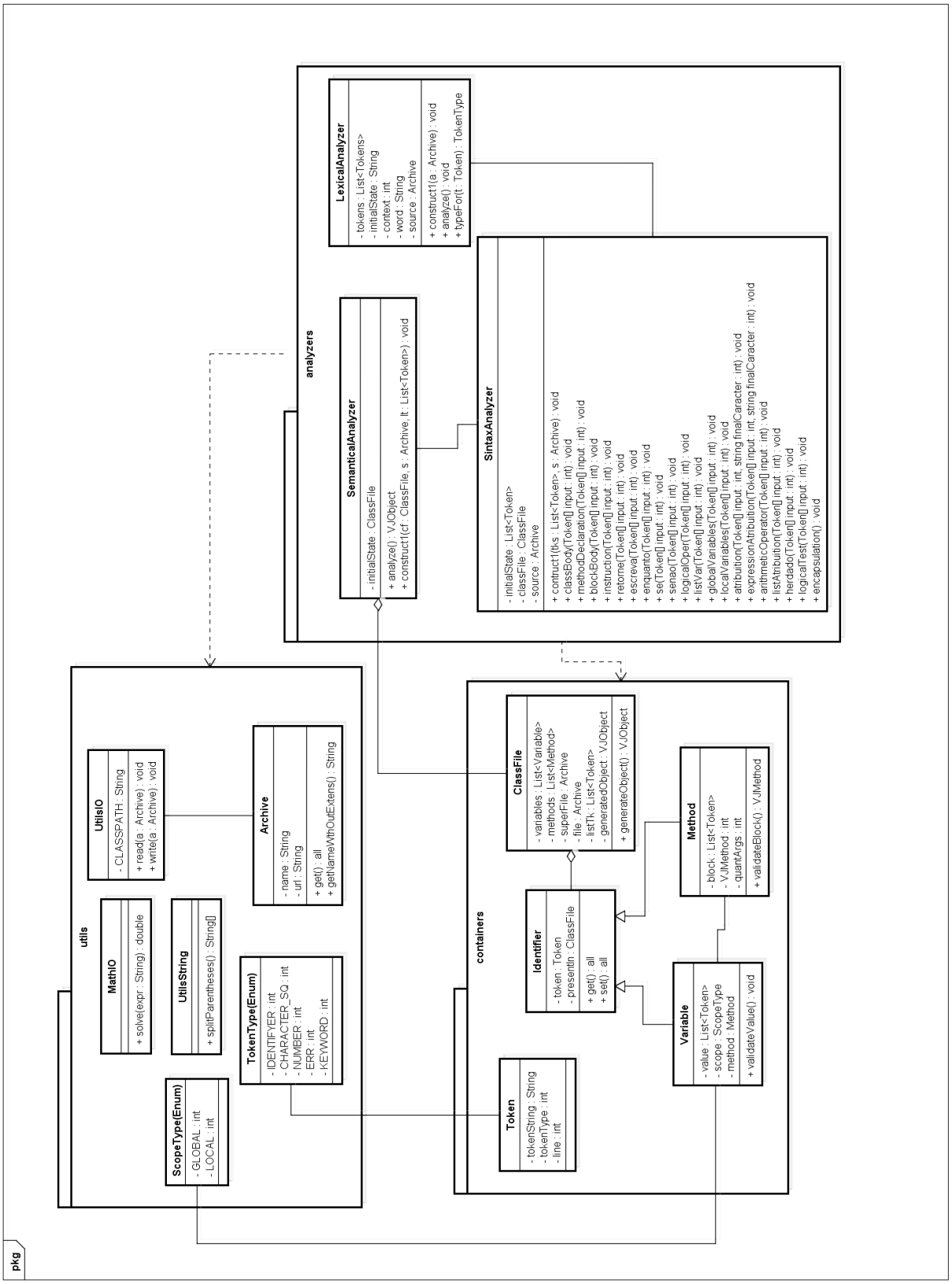
O pacote ***utils*** tem uma função mais auxiliadora, contendo classes responsáveis por realizar operações de limpeza de *strings*, como é o caso da ‘UtilsString’, realizando operações matemáticas com as variáveis numéricas da VJL, caso da ‘MathIO’, ou também fazer operações de entrada e saída de arquivos de texto, função essa das classes ‘Archive’ e ‘UtilsIO’. Esse pacote também possui as classes do tipo ‘Enum’, usadas para uma melhor classificação de certos estados que o interpretador possa necessitar, como a tipagem dos

tokens que o analisador léxico encontra, e também o escopo de variáveis que o analisador semântico verifica.

O pacote ***containers*** contem classes usadas como abstrações de uma linguagem de programação como, por exemplo, ‘ClassFile’, ‘Method’ e ‘Variable’. Objetos dessas classes são gerados após a execução das analises, e utilizados para a execução do código VJL.

Por fim existe o pacote ***analizers***, que contêm as classes responsáveis por fazer de fato as analises léxica, sintática e semântica do código escrito pelo programador.

Figura 14: Classes para análise de códigos VJL



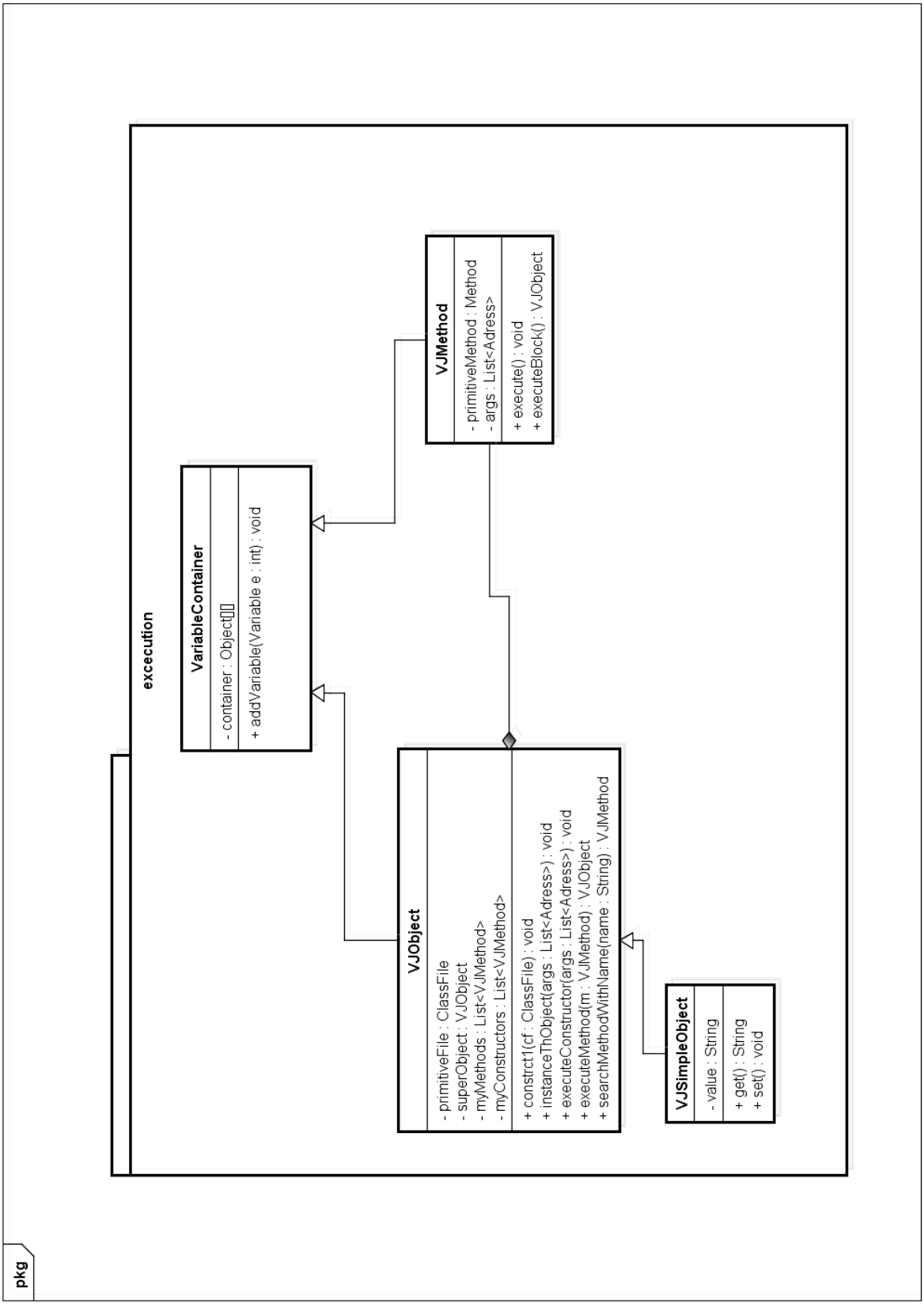
Fonte: Autores (2013)

Outro diagrama de classe criado foi o responsável pelo planejamento da execução do código da VJL, e representa objetos usados para abstrair o nível mais alto da execução de um programa. Esses objetos são abstrações de próprios objetos e métodos da VJL.

Observa-se com este diagrama final que a estrutura do interpretador VJL esta dividido em camadas:

- **Camada de Hardware:** Abstração do nível mais baixo da maquina.
- **Camada de Análise:** Abstração dos analisadores do interpretador, responsáveis pela validação do código escrito.
- **Camada de Execução:** Responsável pela execução do programa em si, faz uso dos objetos gerados na camada de analise e utiliza a camada de hardware para fazer a execução do código, armazenando variáveis e fazendo os cálculos necessários para dar ao programador respostas coerentes.

Figura 15: Classes responsáveis pela interpretação dos comandos VJL



Fonte: Autores (2013)

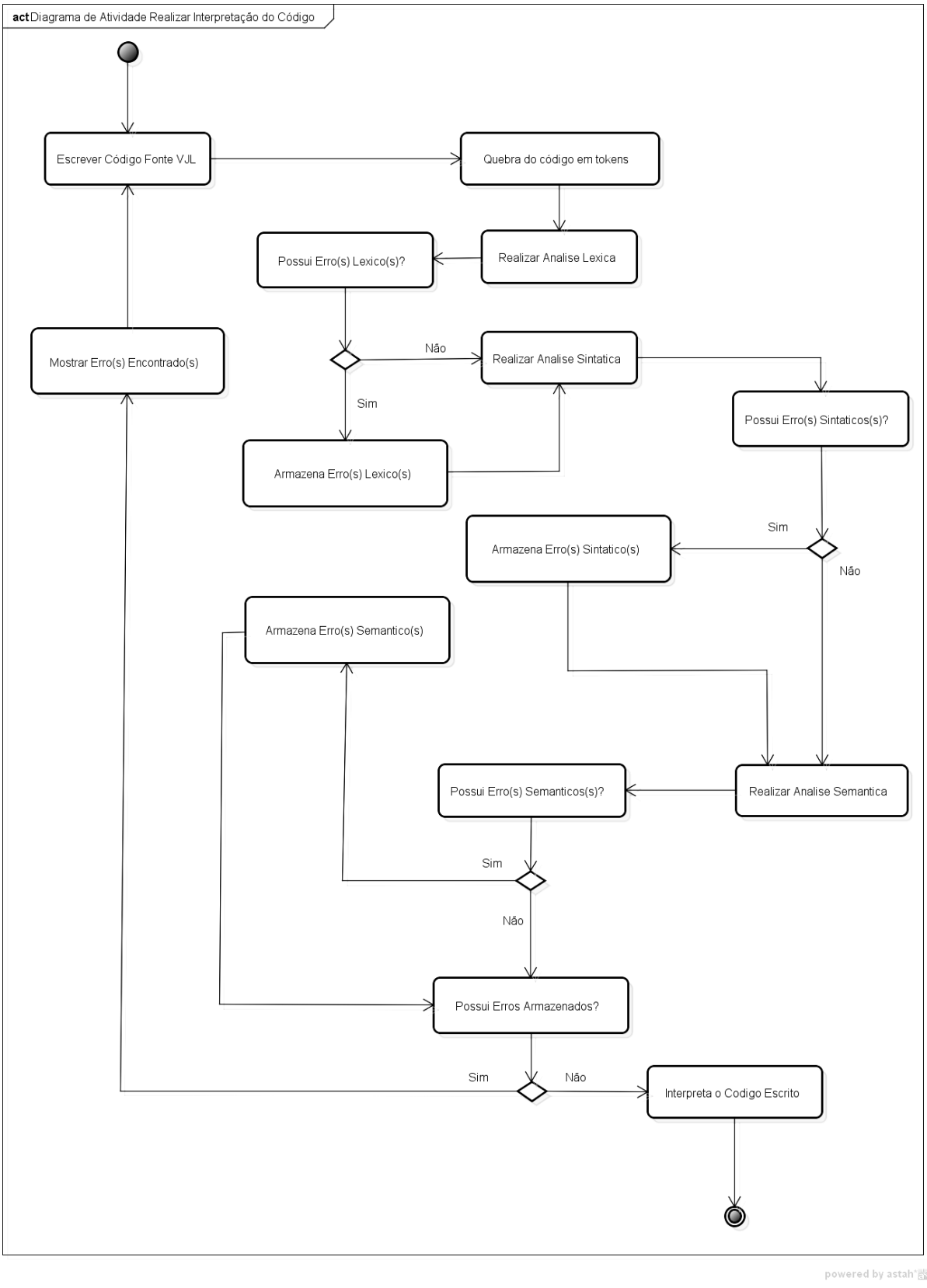
4.2.4.3 Diagrama de Atividade

Para Ramos (2006) o diagrama de atividade pode ser entendido como o tradicional fluxograma, ou seja, é responsável por dar uma visão mais simplificada do fluxo de controle do sistema.

O mesmo autor também afirma que o diagrama de atividade é um dos diagramas que facilitam a compreensão dinâmica do sistema, auxiliando tanto programadores a sincronizar as chamadas de métodos e os acessos às classes, quando analistas de sistemas que devem por uma ordem nos requisitos coletados com os clientes.

O diagrama mostrado na Figura 14 representa o fluxo de atividades que o interpretador deve seguir. Nele são mostradas as principais atividades que ele faz a fim de executar o programa escrito pelo usuário, ou exibir os erros gramaticais que o usuário cometeu ao escrever seu programa.

Figura 16: Diagrama de Atividade do interpretador VJL



Fonte: Autores (2013)

4.3 O JOGO

Com o interpretador desenvolvido, a etapa que falta é somente o escopo onde ele será colocado, que é exatamente no jogo. Ele trabalhará em conjunto com os scripts em Javascript e C# para compilar os códigos desenvolvidos no jogo e gerar respostas como alteração de atributos, criação e edição de novas armas e ações a personagem possa tomar após a execução como a própria fala.

Nosso trabalho apresenta apenas a primeira fase do que viria a ser o jogo completo, a BIOS. Ela é o ambiente que será abordado como o mais primitivo do jogo e onde a aventura da personagem principal terá seu início.

No desenvolvimento de um jogo por completo, cada elemento deve ser construído separadamente, integrado ao jogo e depois codificado para que aja de acordo com as regras e o equilíbrio que todo o jogo necessita ter. Tais com:

- Elementos Visuais
- Sistema de atributos e outros valores
- Lógicas de combate
- Cenas especiais, Legendas e Dublagem.
- Musicalidade

4.3.1 Elementos Visuais

Para a construção visual do jogo, foi preciso aprender modelagem, animação e texturização de objetos tridimensionais. Tomando como exemplo a protagonista e alguns inimigos, será possível observar que a construção dos modelos e com algumas técnicas padrões é uma etapa fundamental para que o projeto não tenha que passar por vários ciclos de readaptação.

4.3.1.1 Modelos 3D

Como já falado neste trabalho a ferramenta usada para modelagem de objetos de dentro do jogo foi a *Blender*, que com uma interface descomplicada facilitou o processo de modelagem da arte conceito 2D para um modelo 3D. A seguir alguns exemplos de modelos feitos na ferramenta seguidos de sua arte conceito.

Figura 17: Inimigo 'Tank', arte conceito e modelo respectivamente

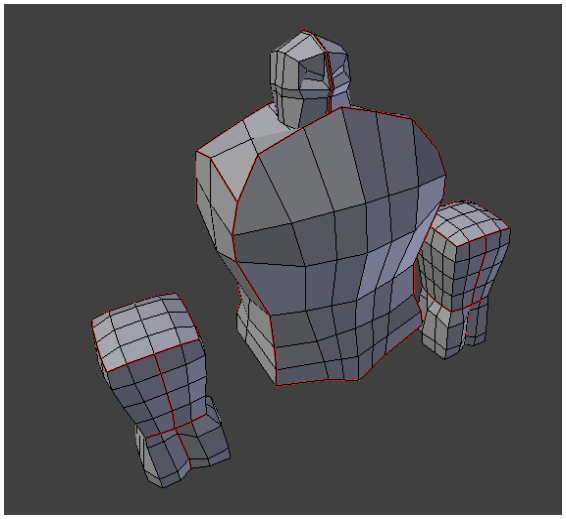


Fonte: Autores (2013).

4.3.1.2 Texturização

Para a texturização foi utilizado tanto o *Blender* para fazer um mapeamento UV[6], usando um método chamado *Mark Seam*[7] da própria ferramenta e exportando o mesmo para a edição na ferramenta de edição gráfica, como pode-se ver na Figura 16.

Figura 18: Aplicação de *Mark Seam* no modelo

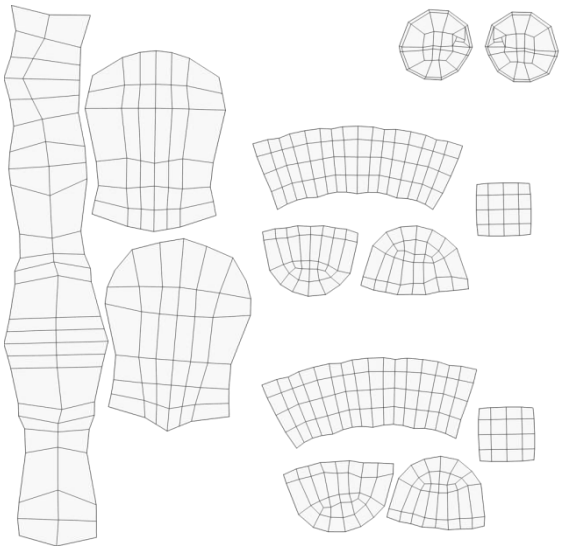


Fonte: Autores (2013).

[6] Mapeamento UV: processo em que a malha de um modelo tridimensional (X, Y & Z) é aberta em um plano bidimensional (X & Y, onde será representado por dois vetores respectivamente, "U & V") (BLENDER WIKI, [2013b?]).

[7] Ferramenta para mapeamento UV que define pontos de corte para obter um plano mais sedimentado. (BLENDER WIKI, [2013c?])

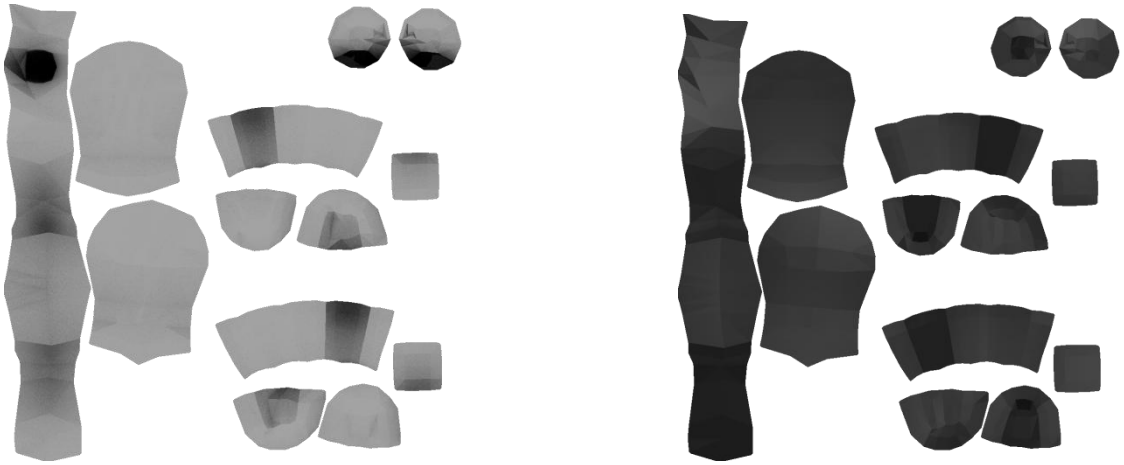
Figura 19: UV pronto para a aplicação da textura



Fonte: Autores (2013).

Tomando como referencia o modelo padrão usado pela Valve, no seu jogo DOTA 2, para texturização de seus personagens e objetos. Nesse modelo existem dois tipos de processos que são o *Ambient Occlusion*[8] e o *Point Light Map*[9], que vão dar um pouco mais de realismo ao modelo.

Figura 20: *Ambient Occlusion* e *Point Light Map* respectivamente



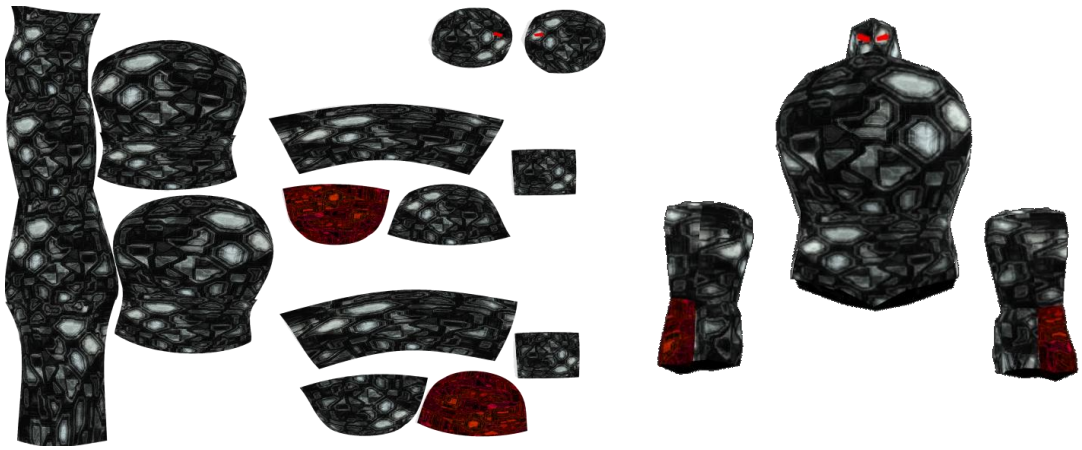
Fonte: Autores (2013).

[8]*Ambient Occlusion*: é um cálculo *ray-tracing* sofisticado que simula suaves sombras iluminação global falsificando escuridão percebida nos cantos e nos cruzamentos de malha, vincos e rachaduras, onde a luz ambiente é ocluído ou bloqueadas. (BLENDER WIKI, [2013a?])

[9]*Point Light Map*: é uma forma de representar a iluminação que vai incidir no modelo 3D, fazendo com que o mesmo fique com uma iluminação natural, não dependendo somente a iluminação do ambiente. (VALVE CORPORATION, 2012)

Para finalizar o processo de texturização é aplicada a cor ou uma imagem, acrescentando os detalhes necessários para que o modelo fique de acordo com os padrões do jogo em questão.

Figura 21: Textura e modelo final

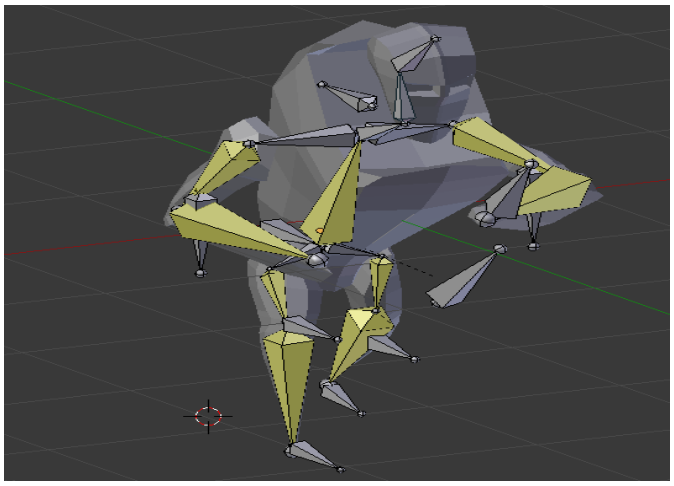


Fonte: Autores (2013).

4.3.1.3 Animação

A animação dos personagens do jogo também foi feito no *Blender*, onde foi feito um ciclo pra cada tipo de animação, como respiração, andar, correr, atacar, entre outros. Para começar foi feito um esqueleto personalizado para cada personagem.

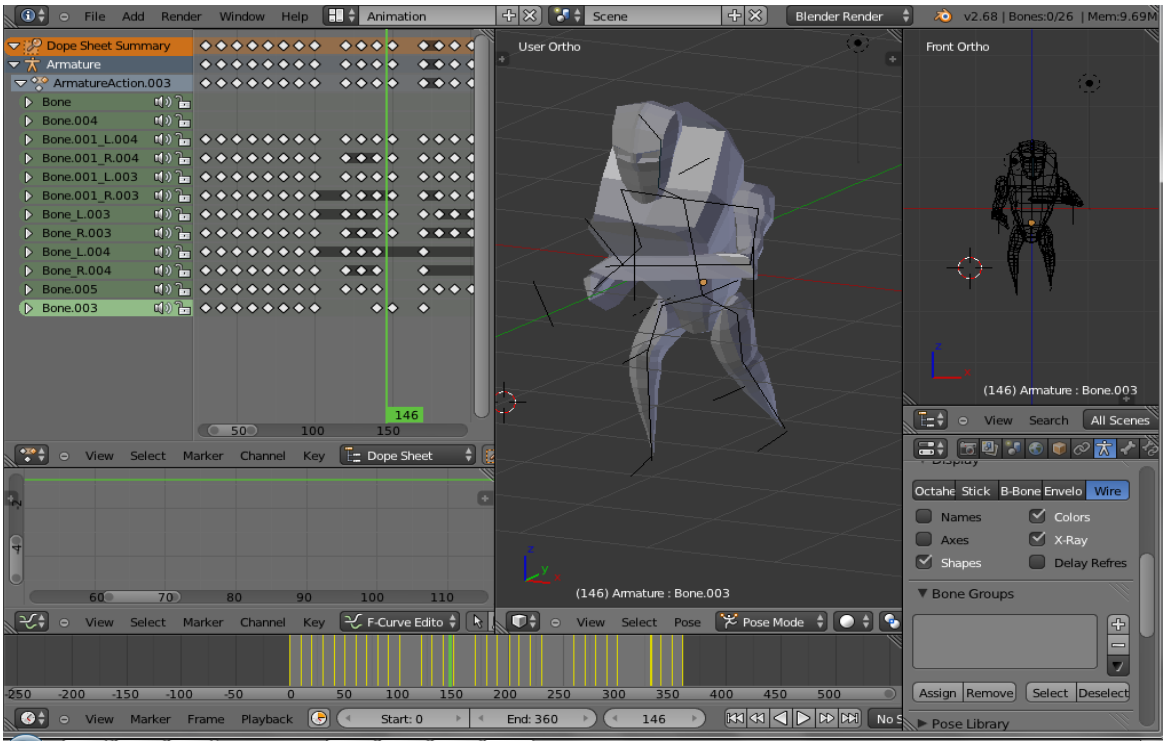
Figura 22: Esqueleto do modelo



Fonte: Autores (2013).

Para fazer o ciclo movimentação do modelo 3D se usa os *Keys Frames* em uma linha do tempo para definir passo a passo os movimentos do modelo.

Figura 23: Ambiente de animação em *Blender*



Fonte: Autores (2013).

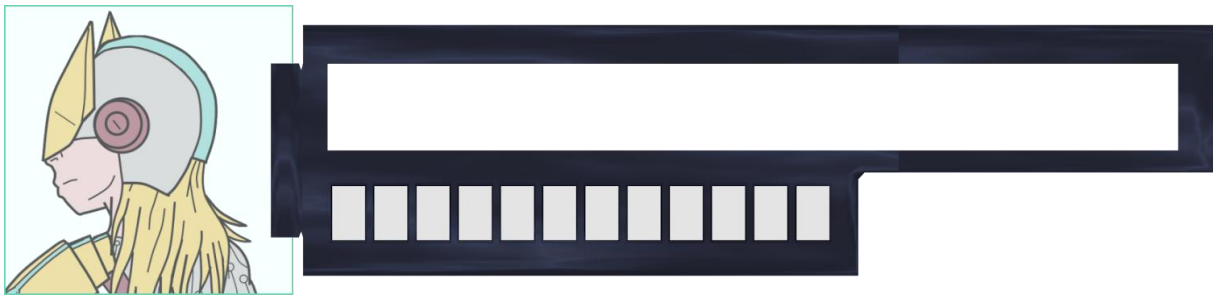
Na Figura 21 pode-se ver no quadro superior esquerdo os *Keys Frames* e mais embaixo a *Timeline*, onde mostra quantos segundos dura a animação.

4.3.1.4 Interface Gráfica.

Outros recursos visuais presentes no jogo são interfaces que facilitem a compreensão que o jogador precisa ter para jogar. A disposição das interfaces principais como barra de vida e legendas, por apresentarem um número pequeno de informações, ocupam espaços menores na exibição do jogo.

No mesmo espaço em que se localiza a vida da personagem também aparece a quantidade de ***Pixel Points*** que a personagem possui sendo vantajoso para o jogador para que o jogador planeje como utilizará seus pontos quando for programar da próxima vez.

Figura 24: Barra de vida

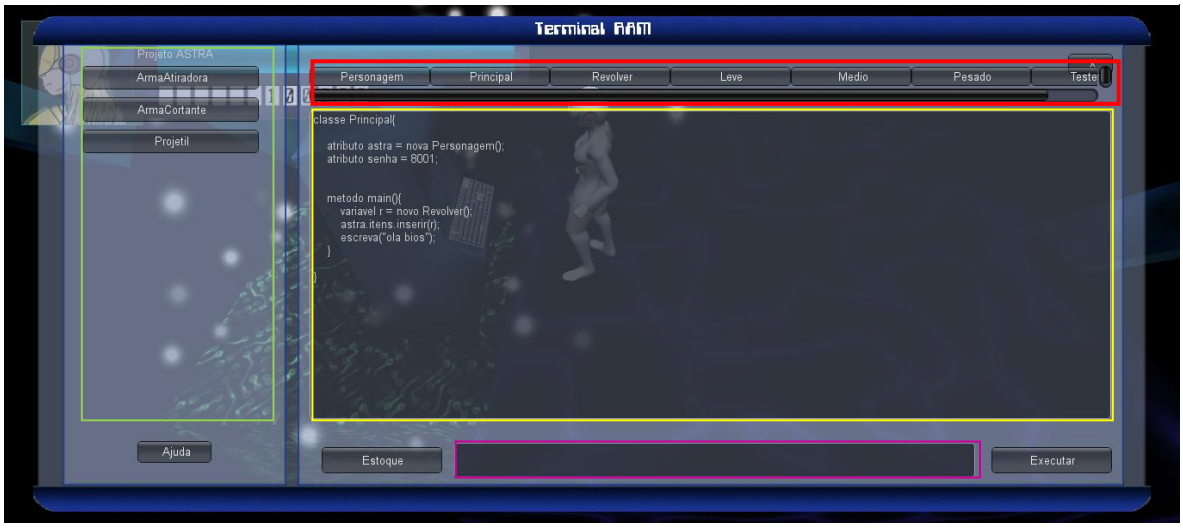


Fonte: Autores (2013).

As interfaces maiores como o tutorial, o estoque e o terminal de programação, que também pode ser chamada de IDE, ocupam toda a tela por disponibilizarem bastante informação e ocultarem elementos que possam distrair a atenção do usuário na hora de programar ou aprender.

A IDE é o ambiente onde ocorre todo o desenvolvimento da personagem, onde os códigos são escritos e gerenciados para que o programa execute de maneira correta e eficiente.

Figura 25: IDE



Fonte: Autores (2013).

Como mostrado na figura acima, o terminal possui um conjunto de ferramentas que podem ser utilizadas para criação e edição de objetos.

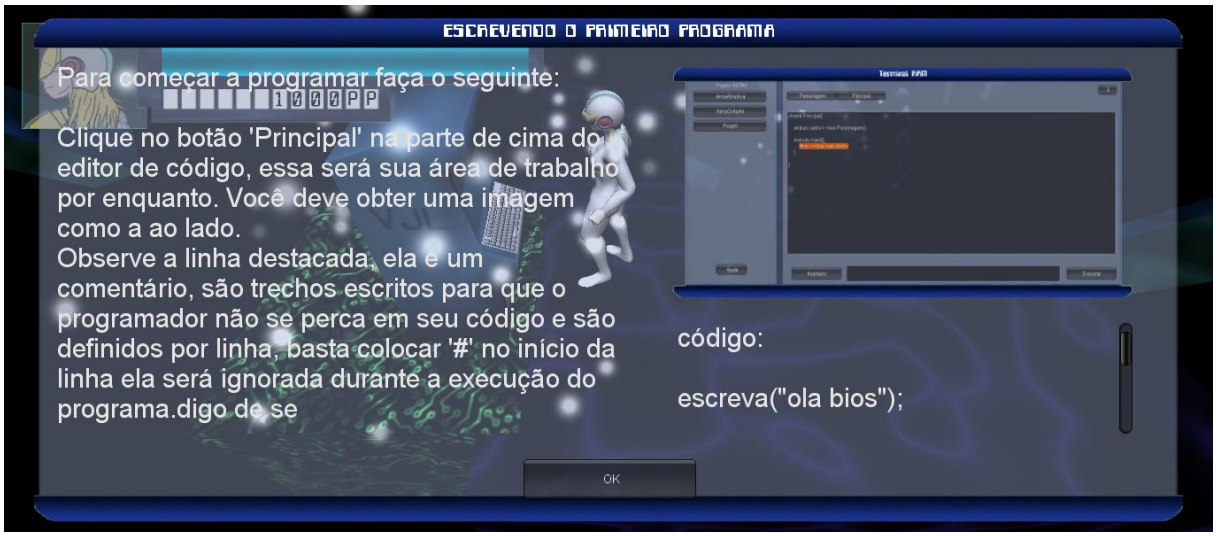
O pacote de classe de verde é o espaço que cria novas classes que herdam os valores do botão que foi clicado. Acima de vermelho temos as abas com todas as classes criadas e presentes na personagem. Clicando em cada aba, aparecerão na área de amarelo os códigos

que serão executados pelo interpretador. Se os códigos estiverem interpretados de maneira correta, as ações no ambiente serão tomadas. Se falhar, ao abrir a IDE de novo será possível visualizar na área lilás chamada console que partes do código foram escritas de maneira errada.

Nessa interface ainda estão presentes três botões, o executar, que é o botão que chama o interpretador para a leitura do código, e os dois botões que levam a outras duas grandes interfaces, o estoque e a ajuda que leva ao tutorial.

O tutorial é outra parte da interface que neste caso tem a única função de proporcionar o conhecimento, a informação desconhecida pelo jogador para aquele momento. A estrutura do tutorial proporciona uma área de texto onde está toda a descrição teórica sobre o tópico, uma área menor com descrição mais prática sobre a aplicação do conteúdo ensinado e uma imagem que auxilie na compreensão do texto.

Figura 26: Janela de tutoriais



Fonte: Autores (2013).

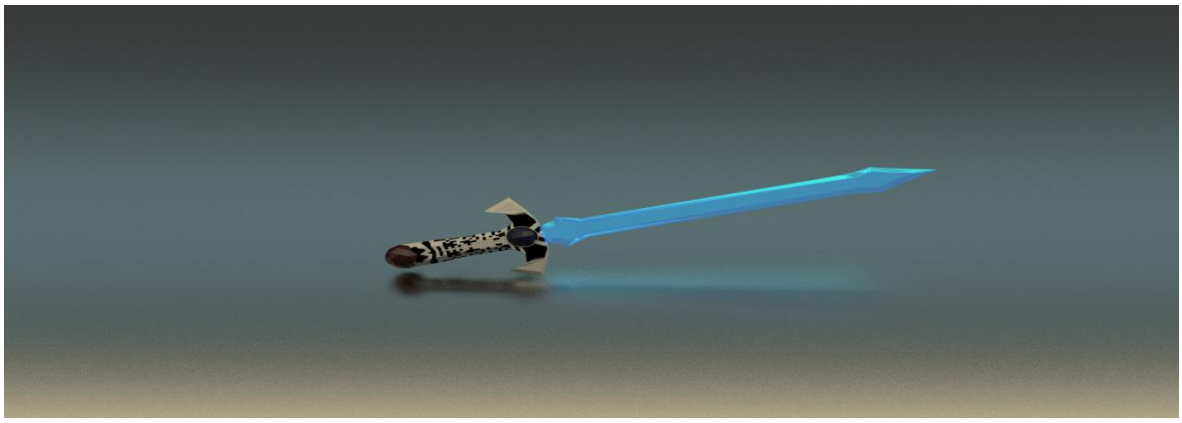
A janela de estoque é onde mostra todos os itens funcionais armazenados na personagem e seus detalhes para que sua compreensão seja mais fácil e rápida. Cada atributo do item gera um detalhamento que define em que o valor está sendo aplicado, assim evitará gastos desnecessários de pontos em atributos que não sejam os desejados pelo jogador.

Para a geração das imagens que ilustram o tutorial e estoque, foi utilizado um recurso do *Blender* para gerar imagens mais atrativas para o tutorial chamado *Cycles*. Criado por Brecht Van Lommel, o *Blender Ray-Tracing*, mas conhecido como *Cycles* é responsável por deixar as características de modelos em 3D mais realistas, é uma ferramenta interna do

próprio *Blender*, tendo como principais características a sua iluminação global bem realista e uma gama de possibilidades à artistas que nele produzem sua renderizações.

No jogo o *Cycles* foi utilizado para renderizar com uma iluminação melhor certos objetos de dentro do próprio jogo, como itens de dentro do inventario, ou objetos explicativos para ajudar o jogador a abstrair ao máximo a ideia passada pelo jogo.

Figura 27: Espada do estoque renderizada por Cycles



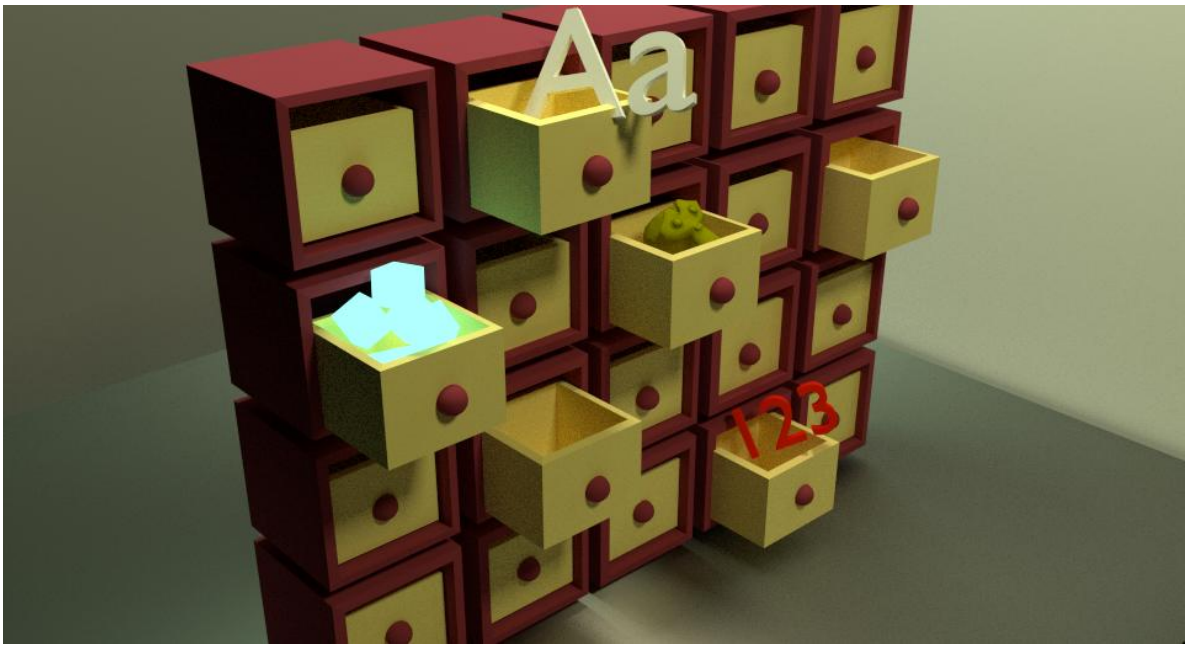
Fonte: Autores (2013).

Figura 28: Pistola do estoque renderizada por Cycles



Fonte: Autores (2013).

Figura 29: Objeto explicativo para janela de tutorial – Memória RAM



Fonte: Autores (2013).

4.3.2 Sistema de Atributos e Itens

Os sistemas em que o jogo se baseou para estruturar o jogo foram inspirados em *Role Playing Game* (RPG). Esses sistemas são populares por gerarem um balanceamento considerável aplicando os cálculos básicos da matemática para determinar as características de cada personagem.

Ao abrir a aba de códigos da classe, são mostrados os quatro atributos básicos da personagem:

- **Força**
- **Agilidade**
- **Destreza**
- **Resistência**

Força influenciará em quanto dano será causado se o ataque for físico, agilidade aumentará a velocidade da personagem e suas animações, destreza aprimorará a força na aplicação do dano no inimigo e resistência ajudará na capacidade da personagem permanecer mais tempo viva.

Itens de um modo geral no jogo são retratados como parte da personagem, seus modelos seriam as classes e a presença deles no jogo seria o mesmo que a instanciação de um

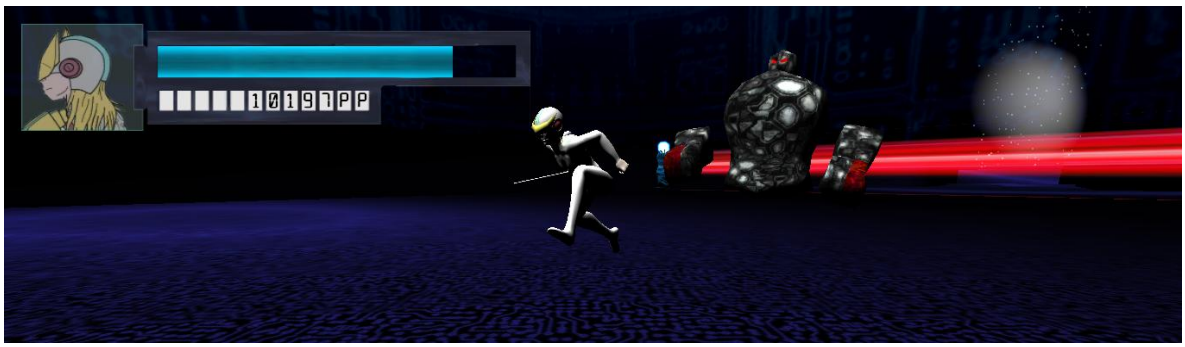
objeto dentro da programação. Poucos foram os itens desenvolvidos, entretanto a linguagem dá suporte à criação de novas armas e equipamentos que trabalhem de maneira única no jogo.

4.3.3 Lógica de Combate

A lógica do jogo consiste em arenas em que os inimigos patrulham ou esperam por emboscadas. Os inimigos são baseados em ameaças virtuais de um computador, por isso ao serem destruídos distribuem ***Pixel Points***. As ações dos inimigos se baseiam em estados de patrulha, perseguição, retorno, investida, ação estratégica e recuo.

No caso da personagem suas ações são tomadas pelo jogador ao pressionar botões do teclado ou mouse. Mais especificamente as ações de combate do jogo se limitam a uma ou mais ações combinadas ao clique do botão esquerdo. De mãos vazias, a personagem faria uma combinação de três ataques se os cliques fossem pressionados no tempo correto. No caso de uma espada, além do alcance de dano ser maior, as ações serão animadas de uma maneira diferente, mas seguindo a lógica dos combates corpo-a-corpo.

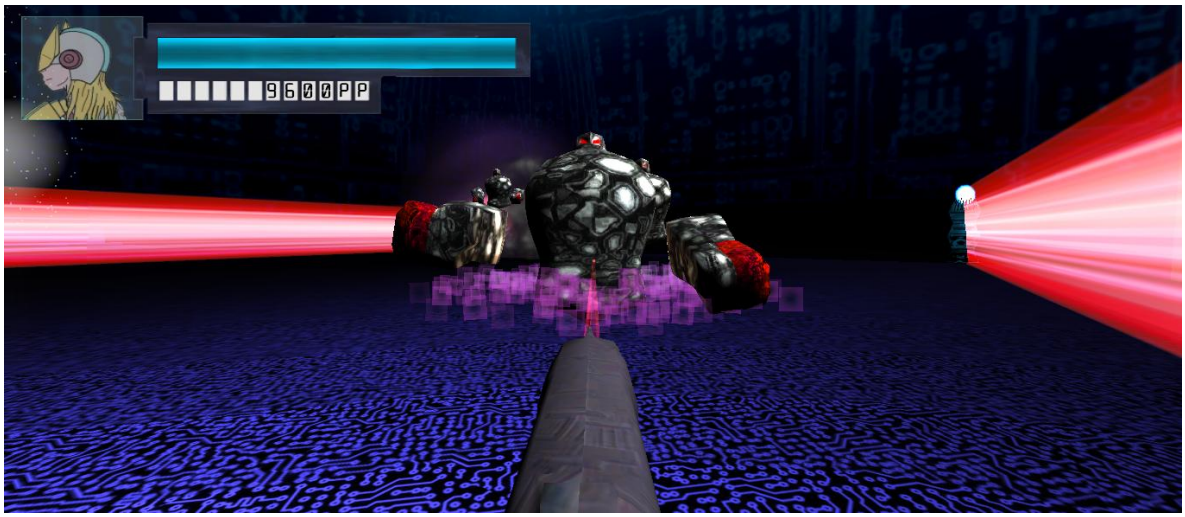
Figura 30: Combate com espada



Fonte: Autores (2013)

O modo FPS citado anteriormente é uma característica específica para itens que tenham a habilidade de atirar. Ao pressionar o botão um dos botões do estoque (0-9), caso o botão pressionado contenha uma arma do tipo atiradora, a tela aproxima para o campo de visão da personagem para aperfeiçoar a capacidade de acertar o tiro. Armas de tiro possuem munição limitada que pode ser comprada através da programação. Cada elemento do jogo procura trabalhar dinamicamente com a linguagem e com o sistema para que o jogo seja compreensível e divertido.

Figura 31: Modo FPS



Fonte: Autores (2013)

4.3.4 Cenas Especiais, Legenda e Dublagem

Por ser um jogo de aventura, o Virtual Justice possui uma história a ser contada. A versão do *Unity* utilizada não permite a utilização de arquivos de vídeo então no caso utilizamos da própria programação para fazer as animações. No cenário existem regiões inteligentes que detectam o momento apropriado para congelar o controle do jogador e fazer a movimentação de alguns objetos. Nesse mesmo tempo são disparadas narrativas baseadas no roteiro desenvolvido na concepção da ideia.

A narrativa é uma combinação de dublagem e legenda. A presença de legenda e uma imagem do personagem que está falando facilitam mais a compreensão da dublagem. Para a dublagem utilizamos do espaço da instituição para gravação, o profissional Hélio Souza para a gravação e edição do áudio e dubladores Thatiana Barros, Regina Mares e Renato Lima para as vozes das personagens. A produção durou duas semanas e foi possível aplicar alguns efeitos de áudio nas vozes para que parecessem mais virtuais com exceção da personagem que deve se manter o mais próximo do humano.

4.3.5 Musicalidade

A musicalidade é um elemento que não pode faltar. Sua presença em um jogo muda completamente a percepção e transforma o jogo em algo envolvente em mais um sentido humano. Por isso, com a ajuda do compositor Neil Turnbull, foi possível produzir músicas de fundo que passassem a essência de exploração e combate para a primeira fase do jogo.

5 CONCLUSÃO

No trabalho desenvolvido foi criado um jogo digital de ensino de técnicas de programação orientada a objetos nomeado Virtual Justice, usando conceitos da metodologia construtivista de Jean Fritz Piaget e do socioconstrutivismo de Lev Semenovitch Vygotsky, o produto que tem como objetivo possibilitar um melhor aprendizado dos conhecimentos por parte do jogador.

Para que o produto desenvolvido tivesse um aspecto lúdico, de forma a agradar o jogador, ele foi criado com base em conceitos de *game design*, focando na experiência que seria passada ao jogador. Como o assunto em questão a ser ensinado era técnicas de programação orientada a objetos, foi criada uma linguagem própria do jogo, batizada de VJL. Desenvolvida pela equipe a VJL possibilitou uma simplificação de conceitos e termos tratados em programação.

Tendo em vista o valor que o desenvolvimento desse projeto agregou ao grupo, a experiência foi de grande valia a todos os integrantes, pois propiciou uma oportunidade de usar em sua grande maioria, os conhecimentos adquiridos nos 4 anos de curso, como gerenciamento e documentação de projetos de software, linguagens e técnicas de programação, geometria analítica. Além da possibilidade de aprendizado de novos conhecimentos ainda não vistos dentro do espaço acadêmico como: modelagem e animação 3D, técnicas de *game design*, metodologias e técnicas pedagógicas assim como novas linguagens de programação como javascript e C#, utilizadas durante o desenvolvimento da também nova ferramenta: *Unity* 3D.

5.1 DIFICULDADES ENCONTRADAS

Durante todo o desenvolvimento do projeto várias dificuldades apareceram. Por ser um jogo, muito do conteúdo necessário para sua implementação partiu de um conhecimento adquirido a parte pela equipe do trabalho. O projeto também passou por várias versões onde o as ferramentas utilizadas eram diferente e a arquitetura e elementos do jogo mudavam até definirmos um escopo e meta.

A criação de uma nova linguagem de programação interpretada pelo C#, a VJL foi um grande desafio para os idealizadores do projeto, pois sua gramática assemelhando-se com a língua portuguesa possibilitava um leque de escritas que tiveram de ser simplificadas, assim como sua própria execução, pois exigia a emulação de uma espécie de memoria RAM para se realizar a alocação de variáveis e métodos.

Fazer a interface entre a linguagem e o jogo foi uma das tarefas mais difíceis devido às possibilidades infinitas que estávamos dispostos a ofertar para o jogador embora o jogo devesse permanecer balanceado pela moeda, outra grande dificuldade. A fundamentação teoria para conseguir explicar as decisões tomadas no design do produto foi buscar um paralelo entre a pedagogia, psicologia e game design, onde dentro do game design deveriam se identificar a presença pedagógica e integrar com a diversão de um jogo.

Outra dificuldade que surgiu durante o desenvolvimento do produto em si foi o fato de ter que trabalhar com diversas ferramentas o que gerou certos contratempos ao se juntar para a criação do projeto final, tais como: a exportação de modelos 3D do *Blender* para a *Unity*, muitas vezes, os modelos não seguiam o mesmo padrão de posicionamento nas duas ferramentas.

5.2 PROPOSTAS FUTURAS

O projeto como um todo teve de ser simplificado para a produção de um trabalho de conclusão, entretanto as aplicações que conseguimos enxergar a partir do que foi construído são tão abrangentes quanto as possibilidades de implementação dentro da própria linguagem VJL. A principal proposta que fica para trabalhos futuros é a finalização do jogo por completo, com todas as fases desenvolvidas e, funcionalidades e recursos aprimorados.

O melhoramento da VJL em nível de execução e algoritmos mais eficientes para as análises léxica, sintática e semântica além da simplificação da gramática para a agilidade no desempenho da linguagem. A adaptação da linguagem VJL em outros jogos educativos de programação também proporcionaria um opções variadas de jogos que alcancem outros públicos, com gostos diferenciados.

Fazer um estudo de caso do produto para avaliar e validar a metodologia e teorias nas quais nos baseamos para o design do jogo. Gerar comprovação estatística e *feedback* dos entrevistados contribuiria para o aperfeiçoamento das técnicas desenvolvidas e usadas no projeto.

A criação de uma comunidade aberta onde seja possível compartilhar ideias e códigos para o desenvolvimento de produtos que trabalhem com os mesmos objetivos de nosso projeto, buscando mostrar o grau de interesse tanto de jogadores como desenvolvedores por uma possível ‘febre’ no desenvolvimento de jogos que permitam a programação de parte de suas funcionalidades.

REFERÊNCIAS

ADOBE SYSTEMS SOFTWARE. **Photoshop CC**, 2013, online. Disponível em: <http://www.adobe.com/br/products/photoshop.html>. Acesso em: 24 nov. 2013.

AHO, A. V.; SETHI, R.; ULLMAN, J. D. Compiladores Princípios, Técnicas e Ferramentas, 1ª Ed. Rio de Janeiro: LTC, 1995.

APOSTILA de compiladores: Universidade Regional Integrada do Alto Uruguai e das Missões. Erechim, 2001. Disponível em: < http://www.cin.ufpe.br/~pftbm/apostila-LFeC-II.pdf>. Acesso em: 19 out. 2013

ARGENTO, Heloisa. **Teoria construtivista**. 2008. Disponível em: <http://www.robertexto.com/archivo5/teoria_construtivista.htm/>. Acesso em: 21 set. 2013.

BLENDER WIKI. **Ambient Occlusion**. [2013a?]. Disponível em: < http://wiki.blender.org/index.php/Doc:2.6/Manual/Lighting/Ambient_Occlusion>. Acesso em: 24 nov. 2013.

BLENDER WIKI. **UV Mapping**. [2013b?]. Disponível em: < http://wiki.blender.org/index.php/Doc:2.6/Manual/Textures/Mapping/UV>. Acesso em: 24 nov. 2013.

BLENDER WIKI. **Edges Tools**. [2013c?]. Disponível em: < http://wiki.blender.org/index.php/Doc:2.4/Manual/Modeling/Meshes/Editing/Edges>. Acesso em: 24 nov. 2013.

BOCK, A. M. B; FURTADO, O; TEIXEIRA, M. de L. T. **Psicologias uma Introdução ao Estudo de Psicologia**.13ª Ed. São Paulo: Editora Saraiva,1999. Disponível em: <http://www.slideshare.net/FelipeMiguel12/psicologias-uma-introducao-ao-estudo-de-psicologia-odair-furtado-maria-de-lourdes-t-ana-merces-bahia-bockwwwlivrosgratisnet>. Acesso em: 21 set. 2013.

CAETANO, L. M. **A epistemologia genética de Jean Piaget**. 2010. Disponível em: <http://www.ip.usp.br/portal/index.php?option=com_content&view=article&id=1797:a-epistemologia-genetica-de-jean-piaget&catid=46:na-midia&Itemid=97>. Acesso em: 29 set. 2013.

CODE. **About Us**. 2013, online. Disponível em: <code.org/about>. Acesso em: 25 nov. 2013.

COREL CORPORATION. **CorelDRAW Graphics Suite X6**, 2013, online. Disponível em: <http://www.corel.com/corel/product/index.jsp?pid=prod4260069>. Acesso em: 24 nov. 2013.

DELL, J. Guess who's winning the brains race, with 100% of first graders learning to code?. Venture Beat, 2012. Disponível em: <http://venturebeat.com/2012/09/04/estonia-code-academy/>. Acesso em: 21 set. 2013

ENTERTAINMENT SOFTWARE ASSOCIATION. **Essential facts about the computer and video game industry**. Washington DC. 2013. Disponível em: < http://www.theesa.com/facts/pdfs/ESA_EF_2013.pdf>. Acesso em: 11 out. 2013.

ESTADÃO. **Mais de 60% dos alunos do ensino superior no País estudam à noite**. 2011, online. Disponível em: <http://www.estadao.com.br/noticias/vidae,mais-de-60-dos-alunos-do-ensino-superior-no-pais-estudam-a-noite,795683,0.htm>. Acesso em: 22 nov. 2013.

FERRARI, M. Lev Vygotsky, o teórico do ensino como processo social. Edição Especial. online. 2008. Disponível em: <http://revistaescola.abril.com.br/historia/pratica-pedagogica/lev-vygotsky-teorico-423354.shtml>. Acesso em: 21 set. 2013.

FILHO, M. L. S. Relações entre Aprendizagem e Desenvolvimento em Piaget e em Vygotsky: Dicotomia ou Compatibilidade?. **Diálogo Educacional**, Curitiba, PR, v. 8, n. 23, p. 265-275, jan./abr. 2008.

GROS, B. Digital Games in Education: The Design of Games-Based Learning Environments. **Journal of Research on Technology in Education**, online, v. 40, n. 1, p. 23-38, 2007.

HUTZ, C. S. et al. O desenvolvimento de marcadores para a avaliação da personalidade no modelo dos cinco grandes fatores. **Psicol. Reflex. Crit.**, Porto Alegre, RS v. 11, n. 2, p. 395-411. 1998.

VANDENBERGHE, J. The 5 Domains of Play. In: GAME DEVELOPER CONFERENCE, 12., 2012. San Francisco, California. **Anais…** San Francisco, California: MOSCONE CENTER, 2012. Disponível em: < http://gdcvault.com/play/1015595/The-5-Domains-of-Play >. Acesso em: 11 out. 2013.

MARKET WATCH. Unity Technologies Releases Facebook Functionality Update to Unity 4. 2013. Disponível em: <http://www.marketwatch.com/story/unity-technologies-releases-facebook-functionality-update-to-unity-4-2013-03-26> Acesso em: 21 set. 2013.

MATISUI, E. et al. Analogias e metáforas: proposição de uma metodologia para ensinar Ciências numa perspectiva construtivista. In: VII CONGRESSO NORTE E NORDESTE DE PESQUISA E INOVAÇÃO – CONNEPI, 7., 2012, Palmas, Tocantins. **Anais...** Palmas, Tocantins: IFTO, 2012. 6 p. Disponível em: <http://propi.ifto.edu.br/ocs/index.php/connepi/vii/paper/viewFile/2584/1513>. Acesso em: 21 set. 2013.

MELO, Ana Cristina. **Desenvolvendo aplicações com UML 2.2: Do Conceitual à Implementação**. 3ª ed. Rio de Janeiro: BRASPORT, 2010. 322 p. Disponível em <http:// books.google.com.br/books?id=BPVHsG17bAYC>. Acesso em: 02 de nov. de 2013.

MESSINA, L. F; TIEDEMANN, K. B. Avaliação da Memoria de Trabalho em Crianças com Transtorno do Déficit de Atenção e Hiperatividade. **Psicologia USP**, São Paulo, SP, v. 20, n. 2, p. 209-228, abr./jun. 2009.

MICROSOFT. **Recursos do Visual Studio**, 2013, online. Disponível em: <http://msdn.microsoft.com/pt-br/vstudio/cc136611.aspx> Acesso em: 24 nov. 2013.

RAMOS, Ricardo. Treinamento Prático em UML: Desenvolva e Gerencie seus Projetos com essa Sensacional Ferramenta. São Paulo: Digerati Books, 2006. 144 p. Disponível em <http://books.google.com.br/books?id=cE4qBWwJM1sC>. Acesso em: 02 de nov. de 2013.

ROSA, T.C. et al. O conceito de zona desenvolvimento proximal no processo de ensino-aprendizagem em sala de aula. In: 21º CONGRESSO DE INICIAÇÃO CIENTÍFICA – CIC, 21. , 2012, Pelotas, Rio Grande do Sul. **Anais...** Pelotas, Rio Grande do Sul: UFPelotas, 2012. 4 p. Disponível em: <http://www.ufpel.edu.br/cic/2012/anais/pdf/CH/CH_00530.pdf/>. Acesso em: 21 set. 2013.

SAVI, R; ULBRICHT, V. R. Jogos digitais educacionais: benefícios e desafios. **Revista Novas Tecnologias na Educação**, online, v. 6, n. 2, 2008.

SEBRAE. **Brasil tem o maior mercado de games no mundo em 2012**. 2013, online. Disponivel em: <http://www.sebrae2014.com.br/Sebrae2014/Alertas/Brasil-tem-o-maior-mercado-de-games-no-mundo-em-2012#.Uo7sfsSkpHT> Acesso em 22 nov. 2013.

SCHELL, J. **A Arte de Game Design: O Livro Original**. Traduzido por Edson Furmankiewicz. Rio de Janeiro: Elsevier, 2011.

SIMON, E. N. **Beyond edutainment exploring the educational potential of computer games.** Copenhagen, 2005. Disponivel em: <http://www.itu.dk/people/sen/papers/egenfeldt.pdf>. Acesso em: 21 set. 2013.

TAKASHIMA, R. S. **Software educacional de auxilio no ensino de sistemas operacionais: uma maneira pratica e dinâmica de aprendizado**. Belém, 2004. Trabalho de Conclusão de Curso (Processamento de Dados) – Centro Universitário do Pará – CESUPA, 2004.

THORNBURG, D. D. **Why STEM Topics are Interrelated: The Importance of Interdisciplinary Studies in K-12 Education**. 2008. Disponível em <http://www.tcse-k12.org/pages/stem.pdf>. Acesso em: 10 jan. 2002.

THROOP, B. **DETONATOR – EXPLOSIONS FOR UNITY3D**. 2009. Disponível em: <http://variancetheory.com/detonator/>. Acesso em: 27 jul. 2013.

UNITED NATIONS, DEPARTMENT OF ECONOMIC AND SOCIAL AFFAIRS. Population Division. 2011. Disponível em < http://populationpyramid.net/WORLD/2010/>. Acesso em: 11 out. 2013.

UNITY TECHNOLOGIES. **Manual - Getting started with Mono Develop**, 2013, online. Disponível em: <http://docs.unity3d.com/Documentation/Manual/HOWTO-MonoDevelop.html>. Acesso em: 24 nov. 2013.

VALVE CORPORATION. **The Evergreen School Visits Valve**. 2011. Disponível em: < http://www.youtube.com/watch?v=QGQo0z3XikI>. Acesso em: 01 out. 2013.

VALVE CORPORATION. **DOTA 2 – Character Color Texture Guide**. 2012. Disponível em:<http://media.steampowered.com/apps/dota2/workshop/Dota2CharacterTextureGuide.pdf >. Acesso em: 06 jul. 2013.

VIRVOU, M; KATSIONES, G; MANOS, K. Combining Software Games with Education: Evaluation of its Educational Effectiveness. **Educational Technology & Society**, v. 8, n. 2, p. 54-65, abr. 2005.

VYGOTSKI, L. S; BRUNER, J. S; MINICK, N. **The Collected Works of L.S. Vygotsky, Volume 1 Cognition and language**. Plenum Press, 1998. Disponível em: <http://books.google.com.br/books?id=fmbimQEACAAJ>. Acesso em: 29 set. 2013.

VYGOTSKI, L. S. et al. **A formação social da mente**. 4ª Ed. São Paulo: Fontes Editora, 1991. Disponível em: <http://www.egov.ufsc.br/portal/sites/default/files/vygotsky-a-formac3a7c3a3o-social-da-mente.pdf>. Acesso em: 21 set. 2013.

APÊNDICE A – MANUAL DE DESENVOLVIMENTO VJL

Manual de Desenvolvimento VJL

INTRODUÇÃO

As explicações que seguem neste documento são referentes a emulação de linguagem de programação denominada ‘VJL’, feita com o objetivo de se parte integrante do jogo educacional de programação ‘Virtual Justice’.

A LP em questão tem por objetivo simular as estruturas e meios que um programador esta acostumado a ver em seus dias de trabalho com linguagens orientadas a objetos, porém com as ações voltadas para o jogo.

Através de um paradigma orientado a objetos será possível o jogador criar seus próprios materiais através de classes VJL pré-definidas sendo possível inserir no jogo através da programação esses objetos e alterá-los conforme a vontade da pessoa que esta programando, como por exemplo através de uma classe VJL chamada Espada o jogador poderia criar realmente uma espada que seria utilizada no decorrer do jogo.

ORIENTAÇÃO

A VJL, como já foi citado acima possui o paradigma POO (Programação Orientada a Objetos) onde é possível se criar objetos com base nas estruturas das classes, formando assim uma melhor visão do mundo real na programação o que é um dos principais objetivos que o jogador/programador veja durante a execução de seu programa. Esse tipo de abstração é o mais utilizado nos dias atuais e portanto foi a escolhida para a VJL.

A LP em questão como toda linguagem orientanda a objetos possui suporte a herança entre as classes o que é essencial para o bom funcionamento do jogo, porém como forma de simplificar a escrita para os mais iniciantes VJL não possui tipagem obrigatória o que a deixa parecida com linguagens de script como Javascript ou mesmo php. VJL possui a chamada tipagem dinâmica, pois variáveis e parâmetros não tem tipos declarados e podem ser associados a objetos de qualquer tipo no tempo de execução (RAMALHO, 2011).

SINTAXE

O desenvolvimento da linguagem tentou simplificar ao máximo a escrita de um programa para iniciantes, possuindo termos em português o que facilita o aprendizado de termos comuns a um programador tais como ‘atributo’, ‘variavel’ e ‘metodo’. A seguir seguem exemplos de estruturação da linguagem.

ESTRUTURAÇÃO DE CLASSES VJL.

As classes ou estruturas na VJL são definidas com a palavra-chave classe e devem possuir o mesmo nome do arquivo ao qual estão sendo escritos sendo assim só é possível escrever uma classe por arquivo.

Exemplo:

```
classe MinhaClasse {
            ...
            ...
}
```

No exemplo acima temos a definição de uma classe denominada ‘MinhaClasse’ e ela deve ser escrita em um arquivo de texto com o mesmo nome, e esse nome não pode incluir espaços em branco, pois são considerados como separadores.

A herança também é uma forma de definir uma classe, VJL não suporta herança múltipla sendo possível apenas uma classe como super classe. Para se definir uma herança entre as classes é utilizada a palavra-chave herda.

Exemplo:

```
classe MinhaSuperClasse{
        ...
    ...
}

classe MinhaSubClasse herda MinhaSuperClasse{
        ...
        ...
}
```

No exemplo acima temos a super classe ‘MinhaSuperClasse’ e a sub classe MinhaSubClasse os atributos e métodos da super classe passam a ser validos também para a sub classe.

Como todas as classes em POO elas devem possuir os chamados construtores que são os métodos que serão executados ao se rodar uma instanciação da classe. Para definir construtores é usada a palavra-chave ‘construtor’. Caso queira passar algum argumento veja o exemplo abaixo.

Exemplo

```
classe MinhaClasse {

        construtor MinhaClasse(){

        }

        construtor MinhaClasse(variavel v){

        }

}
```

Em classes que herdam de outras o construtor é de extrema importância, pois a cada instanciação da subclasse uma instanciação da superclasse é requerida, caso essa super classe tenha construtores diferentes eles precisão ser passados da subclasse para a super classe através da palavra chave herdado sempre na primeira linha do construtor.

Exemplo

```
classe MinhaSuperClasse{

    construtor MinhaSuperClasse(variavel arg){

    }
}

classe MinhaSubClasse herda MinhaSuperClasse{

      construtor MinhaSuperClasse(variavel argumento){
              herdado(argumento);
      }

      #ou...
      construtor MinhaSuperClasse(){
              herdado(10);
      }

}
```

ESTRUTURAÇÃO DE ATRIBUTOS E VARIÁVEIS

Cada classe em VJL possui os chamados atributos que nada mais são do que as chamadas variáveis globais que definem as características do objeto gerado pela determinada classe os atributos são definidos no corpo da classe apenas e para defini-los é usado a palavra-chave atributo.

Exemplo

```
classe MinhaClasse{

        atributo primeiroAtributo = 10;
        atributo seguntoAtributo = “definição”;
        atributo terceiroAtributo = novo Classe1();
        atributo quartoAtributo = nova Classe2();
        atributo quintoAtributo = nada;

}
```

Nota-se que não existe uma tipagem e sim a sua definição no tempo de execução.

NOTA [1]: A definição para a instanciação de objetos segue no exemplo acima, é utilizada as palavras-chave novo ou nova conforme a vontade do programador seguido do nome da classe que esta se querendo instanciar, caso a classe tenha construtores os argumentos são passados via os ‘(...)’ ao lado do nome da classe

NOTA [2]: Observe que o quinto atributo é uma palavra-chave **nada** ela representa valor nulo, não é objeto não é número nem cadeia de caractere pode apenas ser usada para guardar espaço de memória que será usado posteriormente ou para fazer testes lógicos e atribuições.

ESTRUTURAÇÃO DOS MÉTODOS

Os métodos são como se fossem funções e ações que os objetos podem executar em VJL essas estruturas são declaradas de forma semelhante aos construtores das classes, através da palavra-chave metodo dentro do corpo da classe.

Exemplo

```
classe MinhaClasse {

        metodo meuMetodo(){

        }


        metodo meuMetodo(variavel argumento){

        }

        metodo meuMetodo(variavel argumento, variavel argumento2){

        }

         metodo maisUmMetodo(){

         }

}
```

Os métodos podem ter argumentos passados a ele através de passagem de variáveis pelos “( )” e são diferenciados um dos outros pelo seu nome e pela quantidade de argumentos.

Os métodos também podem ter retornos opcionais que fica a cargo do programador esse artifício é feito pela palavra chave retorne.

Exemplo

```
classe MinhaClasse {

        metodo meuMetodo(){
                retorne 10+10;
        }

        metodo meuMetodo(variavel argumento){
              se(argumento < 8){
                        retorne argumento + 10;
              }
        }

}
```

Note que o primeiro método sempre vai retornar alguma coisa no caso: 20, porém o segundo não necessariamente retorna algo, em caso de um método estar sendo usado em uma atribuição e esse método não retornar nada o valor representado será nulo, em VJL todo e qualquer tipo de valor nulo é representado pela palavra-chave nada.

Exemplo

```
classe MinhaClasse {

        metodo meuMetodo(){
                retorne 10+10;
        }

        metodo meuMetodo(variavel argumento){
                se(argumento < 8){
                        retorne argumento + 10;
                }
        }

         metodo maisUmMetodo(){

                  variavel valorNulo = meuMetodo(10);
                 # O valor da variavel acima agora é nada

                 variavel valorConteudo = meuMetodo(5);
                  # O valor da variavel acima agora é 15
         }

}
```

ESTRUTURAS DE REPETIÇÃO E DE CONTROLE

As estruturas presentes na linguagem são as mais básicas tendo apenas o chamado laço ‘se’ ‘senão’ e o laço de repetição ‘enquanto’ elas devem ser declaradas dentro do corpo dos métodos ou dentro do corpo de construtores.

Exemplo

```
classe MinhaClasse {

        metodo fatoria(variavel n){
                se(argumento <= 1){
                        retorne 1;
                } senao {
                        retorne fatorial(n - 1);
                }
        }
}
```

Exemplo

```
classe MinhaClasse {

        metodo fatoria(variavel n){
                variavel resposta = 1;
                        enquanto(n > 1){
                         resposta = resposta * n;
                         n = n – 1;
                }
                retorne resposta;
        }
}
```

ENCAPSULAMENTO.

Assim como toda linguagem de programação orientada a objetos, a VJL também possui a possibilidade de encapsulamento de variáveis e métodos. Com o encapsulamento os respectivos objetos da linguagem não podem ser acessados de fora da classe de origem dos mesmos, ou seja, um método ou um atributo privado podem apenas ser chamados dentro de métodos da mesma classe em que estão presentes.

Exemplo

```
classe Usuario {

        atributo privado senha = “1234”;
        atributo publico nome = “nome”;
        atributo idade = 10;

}
```

O exemplo acima é de uma classe com 3 atributos, dos quais 1 é privado e os outros dois são públicos.

Para declarar um atributo como publico, basta usar a palavra-chave **publico**, e assim o atributo pode ser acessado por outras classes sem a necessidade de métodos getters e setters, outra maneira é não colocar nada ao lado de ‘atributo’, a linguagem toma como padrão que um atributo é público.

Caso o programador queira proteger seus atributos ele pode deixa-los privados dentro da classe, utilizando a palavra-chave **privado**. Nesse caso a modificação do respectivo atributo só pode ser feita no escopo da classe.

Exemplo

```
classe Usuario {

        atributo privado senha = “1234”;
        atributo publico nome = “nome”;
        atributo idade = 10;

        metodo alterarSenha(variavel v){
                senha = v;
        }
}


classe Principal {

        metodo main(){
                Usuario u = novo Usuario();
                u.alterarSenha(“5432”); # Maneira correta
                u.senha = “5432” # Gera erro de semântica
        }

}
```

O encapsulamento também pode ser aplicado em métodos, da mesma forma que é aplicado nos atributos de classe, com a utilização das palavras-chave **publico** e **privado**, assim como nos atributos a ausência dessas palavras implica dizer que o método é publico.

Exemplo

```
classe Usuario {

        atributo privado senha = “1234”;
        atributo publico nome = “nome”;
        atributo idade = 10;
        atributo privado login = “login”;

        metodo publico alterarSenha(variavel v){
                senha = v;
        }

        metodo privado calcularSalario(variavel x){
                retorne x + 100;
        }

        metodo alterarLogin(variavel lo){
               login = lo;
        }

}
```

```
classe Principal {

        metodo main(){
                Usuario u = novo Usuario();
                u.alterarSenha(“5432”); # Maneira correta
                u.alterarLogin(“novo login”); # Maneira correta

                u.calcularSalario(1000); # Gera erro de semântica

        }

}
```

COMANDO ESCREVA E COMENTÁRIOS

A VJL possui um comando nativo, é o **escreva** que permite você mostrar informações na tela de saída correspondente.

Exemplo

```
classe MinhaClasse {

        construtor MinhaClasse(){
                escreva(fatorial(3));
        }


        metodo fatoria(variavel n){
                se(argumento <= 1){
                  retorne 1;
            } senao {
                  retorne fatorial(n - 1);
            }
        }
}
```

No exemplo acima ao se instanciar a classe temos a saída ‘6’ pois o método será executado obtendo o valor do fatorial de 3 (3*2*1 = 6).

Um artifício importante para o programador são os comentários da linguagem que foram vistos durante alguns dos exemplos acima, eles são definidos pelo caractere ‘#’ e terminam com uma quebra de linha.

Exemplo

```
#Exemplo de comentario
classe MinhaClasse {

        metodo facaAlgumaCoisa(){
            # comentários podem ser escritos em qualquer parte
            # do código do programados, eles são excluídos da
            # execução do programa.
        }

}
```

REFERÊNCIAS

RAMALHO, L. **Introdução à Orientação a Objetos em Python (sem sotaque)**. Disponível em: < http://www.slideshare.net/ramalho/orientao-a-objetos-em-python-presentation> Acesso em: 6 jul. 2013.

APÊNDICE B – Detalhamento de Caso de Uso para Análise Léxica.

Nome	Realizar análise léxica.
Descrição	Possibilita o sistema de dar uma resposta quanto a assertiva de léxica da linguagem
Objetivo	Obter uma resposta para a léxica do programa
Atores	Programador
Pré Condição	-Nenhuma-
Dados de Entrada e Saída	**Entrada**: o arquivo de classe, o seu nome e a url **Saída**: uma lista de Tokens que carregam as informações de linha, classe e tipo de cada palavra escrita pelo programador
Sequência de Troca de mensagens	
Ator	**Sistemas**
1. Solicita a compilação de seus códigos.	2. Quebra o texto fornecido em tokens de acordo com as normas léxicas que a linguagem possui, classificando cada sentença 3. Percorre a lista de tokens obtida para verificar se existe algum token não identificado como palavra chave, identificador, sequência de caractere ou número.
Curso alternativo – Não possui-	
Ator	**Sistema**
	3. Caso encontre um desses tokens defeituosos o sistema gera uma mensagem de alerta informando que um erro léxico foi encontrado.
Pós condição	---Não Possui---
Interfaces	Interface de IDE

APÊNDICE C – Detalhamento de Caso de Uso para Análise Semântica.

Nome	Realizar análise semântica.
Descrição	Possibilita o sistema de dar uma resposta quanto a assertiva semântica da linguagem
Objetivo	Obter uma resposta para a semântica do programa
Atores	Programador
Pré Condição	-Nenhuma-
Dados de Entrada e Saída	**Entrada**: objeto ClassFile gerado pela analise sintática. **Saída**: o objeto código, VJObject que representa a classe compilada.
Sequência de Troca de mensagens	
Ator	**Sistemas**
1. Solicita a compilação de seus códigos.	2. Faz a verificação de cada variável e cada bloco de método para procurar por erros semânticos.
Curso alternativo -Não possui-	
Ator	**Sistema**
	2. Caso encontre algum desses erros mostra uma mensagem informando qual a variável e qual o método e a classe em que foi encontrado esse erro.
Pós condição	---Não Possui---
Interfaces	Interface de IDE

APÊNDICE D – Detalhamento de Caso de Uso para Análise Sintática.

Nome	Realizar análise sintática.
Descrição	Possibilita o sistema de dar uma resposta quanto a assertiva sintaxe da linguagem
Objetivo	Obter uma resposta para a sintaxe do programa
Atores	Programador
Pré Condição	-Nenhuma-
Dados de Entrada e Saída	**Entrada**: a lista de tokens obtida da analise léxica **Saída**: o objeto ClassFile para posteriormente ser usado pela analise semântica.
Sequência de Troca de mensagens	
Ator	**Sistemas**
1. Solicita a compilação de seus códigos.	2. Analisa as sentenças de tokens e a cada método como por exemplo a declaração de método ele vai criar objetos como por exemplo ‘Method’.
Curso alternativo –Não possui-	
Ator	**Sistema**
	2. Caso algum dos métodos gere uma exceção de sintaxe o programa manda a mensagem correspondente com uma sugestão para conserto da sentença
Pós condição	---Não Possui---
Interfaces	Interface de IDE

APÊNDICE E – Detalhamento de Caso de Uso para Executar O Programa.

Nome	Executar o seu programa
Descrição	Possibilita que o usuário possa ver a execução de seu código em uma saída de execução
Objetivo	Emular uma memória virtual e fazer com que as classes, variáveis e método declarados pelo programador possam ser executadas
Atores	Programador
Pré Condição	A compilação do código escrito pelo programador, feita nas funcionalidades: AnaliseLexica, AnaliseSintatica e AnaliseSemantica.
Dados de Entrada e Saída	**Entrada**: o objeto VJObject principal criado após a analise semântica. **Saída**: execução em si do programa feito pelo programador
Sequência de Troca de mensagens	
Ator	**Sistemas**
1. Solicita que o programa seja validado. 2. Após obter uma resposta positiva quanto a validação ele solicita a execução	3. Executa métodos para que cada variável seja armazenada em uma memória virtual que emula a de uma maquina de verdade. 4. Quebra cada instrução do método principal para que seja executada sequencialmente com o auxilio da memória virtual, que armazena os valores esperados da execução.
Curso alternativo –Não possui-	
Ator	**Sistema**
	4. Caso o sistema encontre algum impedimento devido a um erro lógico ele gera uma exceção a tempo de execução informando a classe e a linha que o erro lógico apareceu, informando também qual é o erro, o seu motivo.
Pós condição	---Não Possui---
Interfaces	Interface de IDE

APÊNDICE F – Detalhamento de Caso de Uso para Validar o Código.

Nome	Validar o Código
Descrição	Possibilita o programador obter uma resposta se seu código esta correto ou não
Objetivo	Dar ao programador uma confirmação de que seu código ira ser executado pela maquina
Atores	Programador
Pré Condição	-Nenhuma-
Dados de Entrada e Saída	Entrada: a pasta com os arquivos de classe (no projeto: 'Archive') que contem os códigos em estado bruto, apenas seu texto
Sequência de Troca de mensagens	
Ator	**Sistemas**
1. Solicita a interface da IDE seja ela qual for: 'txt' ou a IDE do jogo em si. 2. Escreve e salva seus códigos em uma pasta que será fornecida ao sistema.	3. Abre a pasta e faz a execução da analise léxica 4. Faz a execução da analise sintática. 5. Faz a execução da analise semântica.
Curso alternativo	
Ator	**Sistema**
	3. Informa ao programador através da interface qual o erro léxico. 4. Informa ao programador através da interface qual o erro sintático. 5. Informa ao programador através da interface qual o erro semântico.
Pós condição	---Não Possui---
Interfaces	Interface de IDE

APÊNDICE G – Detalhamento de Caso de Uso para Visualizar os Erros.

Nome	Visualizar seus Erros
Descrição	Possibilita o programador ler com mais detalhes os erros obtidos na fase de compilação.
Objetivo	Dar ao programador um melhor entendimento sobre o que saiu de errado com os códigos compilados
Atores	Programador
Pré Condição	O programa ter sido validado
Dados de Entrada e Saída	-Nenhuma-
Sequência de Troca de mensagens	
Ator	**Sistemas**
1. Solicita a compilação de seus códigos.	2. Realiza as validações da fase de compilação 3. Disponibiliza as informações: de linha, classe e uma sugestão para o erro
Curso alternativo –Não possui-	
Ator	**Sistema**
Pós condição	---Não Possui---
Interfaces	Interface de IDE